NetCfg: A Declarative Network Configuration Compiler

Formal Methods in Configuration Synthesis

Simon Knight, Ph.D.

Principal Architect, AutoNetKit Research

March 2026 • Version 2.3.0

Abstract. NetCfg is a Rust-based network configuration compiler that transforms high-level topology intent into verified, target-specific device configurations. The compiler processes intent through a five-layer pipeline: (1) Intent Transformation via composable primitives, (2) Topological Validation via a DSL-driven assertion engine, (3) Lowering to a vendor-neutral Device Intermediate Representation (DeviceIR), (4) Target-Specific AST Transformation (TSDM) for hardware-aware lowering, and (5) Syntax Serialisation via dumb templates. This report documents the data model, the transformation DSLs, and the AST-based lowering mechanism that decouples architectural logic from physical hardware constraints.

Version	Date	Changes
2.3.0	March 2026	Ph.D. Insights & Design Rules; v2.3 Standard.
1.1.0	March 2026	Upgraded to sk-techreport v2.2 standard; Integrated Synthesis Funnel.
1.0.0	Jan 2026	Initial release of the Rust-based configuration compiler.

Contents

List of Figures

List of Tables

List of Design Decisions & Rules

1 Introduction

The configuration of large-scale network infrastructure is essentially a compilation problem.

Insight: Manual CLI entry is the "assembly language" of networking; NetCfg provides the high-level language and compiler necessary for scale.

Shenker [1] diagnosed the fundamental issue: unlike virtually every other discipline in computer science, networking has addressed complexity by adding protocols rather than creating abstractions. The result is a "bag of protocols"—each solving one narrow problem, with no composable interfaces between them. He proposed three abstractions for software-defined networking—a forwarding interface, a global network view, and a specification language—that would decouple intent from mechanism.

Gill et al. [2] quantified the operational cost of this absence: analysing five years of failure data from Microsoft's data centres, they found that misconfiguration accounted for the largest share of customer-impacting incidents, with operator error during maintenance being the dominant trigger. As data-centre networks have since grown to millions of lines of device configuration, the case for automated, intent-driven synthesis has only strengthened—yet most automation remains bespoke and not transferable.

More recently, Google's MALT [3] system introduced *multi-abstraction-layer topology* modelling, demonstrating that a shared topology representation spanning physical, logical, and design layers could unify over 100 internal teams. MALT's key insight—that "almost all other network-management data either derives from its topology, constrains how to use a topology, or associates resources with specific places"—directly motivates NetCfg's graph-centric design.

Meanwhile, model-driven management via OpenConfig [4] and YANG [5] has standardised vendor-neutral *per-device* data schemas, but these models do not capture cross-device topology relationships or support graph-wide operations like IP allocation across edges.

> **: Hardware Independence**
>
> Architectural logic must be lowerable to a vendor-neutral Intermediate Representation (DeviceIR) before any target-specific syntax is generated. This ensures that policy intent remains decoupled from physical ASIC constraints.

What NetCfg is

NetCfg is a command-line tool and Rust library that compiles declarative network topology descriptions into per-device configuration files. It takes a *blueprint* (describing what the network should look like) and a *mapping* (describing how to extract vendor-specific configuration from the topology) and produces deterministic, diffable, auditable configuration artifacts for every device in the network.

NetCfg replaces the manual "glue" layer between source-of-truth systems (NetBox [6], Nautobot [7]), automation frameworks (Ansible [8], Nornir [9]), and network modelling tools (Batfish [10]). It treats network configuration as a *compilation problem*: topology intent is compiled through a typed pipeline, not interpolated through ad-hoc templates. Where OpenConfig standardises the *schema* of device state, NetCfg standardises the *derivation* of that state from topology intent. Where MALT provides a shared topology *representation*, NetCfg provides a topology *compiler* that transforms intent into per-device configuration.

Design philosophy

Three principles explain every design decision in this document:

1. **Lowering over templating.** The operator's intent is lowered through successive stages of abstraction, mirroring the multi-stage lowering architecture of LLVM [11]. Just as LLVM lowers high-level source through a language-independent IR (LLVM IR) to target-specific machine code,

NetCfg lowers topology intent through a vendor-neutral DeviceIR to platform-specific CLI syntax. The analogy is precise: LLVM's frontend/IR/backend separation corresponds to NetCfg's blueprint/DeviceIR/template separation. This ensures that hardware quirks and vendor-specific logic are handled via structured AST transformations, not ad-hoc string concatenation.

2. **Separation of what from how.** The blueprint declares *what* the network should look like (topology transformations). The mapping declares *how* to extract device configuration from the result. The core blueprint remains vendor-neutral; platform-specific rules are injected via `imports: ['hardware-lowering.yaml']` and isolated behind `when: device_os == ...` predicates. Thus, the same blueprint logic can produce Cisco IOS, Arista EOS, or Juniper JunOS output by swapping only the mapping and templates.
3. **Topology as the single source of truth.** Following MALT's insight that topology is the root from which all other network-management data derives, NetCfg treats the graph as the canonical representation. IP addresses, protocol sessions, policy rules, and security zones are all *computed* from topology relationships, not declared independently.

Audience

This document is intended for:

- Engineers integrating NetCfg into a network automation pipeline.
- Contributors to the NetCfg and NTE codebases.
- Researchers evaluating graph-based approaches to network configuration.

Familiarity with YAML, basic graph theory, and IP networking is assumed. Experience with Rust is helpful but not required for the API sections.

How to read this document

§2 introduces the conceptual model—the layered graph, blueprint composition, and the select/enrich/validate pattern—without any code or formalism. §3 then grounds these concepts in the formal data model and Rust implementation. Readers wanting the blueprint syntax should continue to §5 (blueprint DSL) and §8 (CLI reference), referring to individual sections as needed. §6–§7 cover the compilation pipeline and core mechanisms in depth and can be deferred until required. §9 provides a complete end-to-end worked example that traces a blueprint through every pipeline stage. §11 documents the language server and REST API for editor and CI/CD integration.

2 Conceptual Model

Before introducing formal definitions or code, this section explains the three ideas that underpin every NetCfg blueprint: (1) the network model is a *layered graph*; (2) blueprints *compose* the model declaratively, layer by layer; (3) every primitive follows the same *select* → *enrich* → *validate* pattern. Understanding these concepts makes the formal data model (§3) and DSL syntax (§5) easier to absorb.

Keep the mental model, postpone the mechanisms. In this section, treat NetCfg as a compiler operating over a graph: selectors identify where in the graph to act, primitives add derived state, and assertions act as quality gates. The implementation details (Rust structs, CEL compilation, DataFrames) exist to make this model fast and safe, but they are not required to understand the flow.

2.1 A mental model of the compiler

At a high level, NetCfg takes a base topology and progressively enriches it until each device node contains enough derived state to render deterministic, diffable configuration. Figure 1 shows the compilation funnel; Table 1 summarises the artefact produced at each stage.

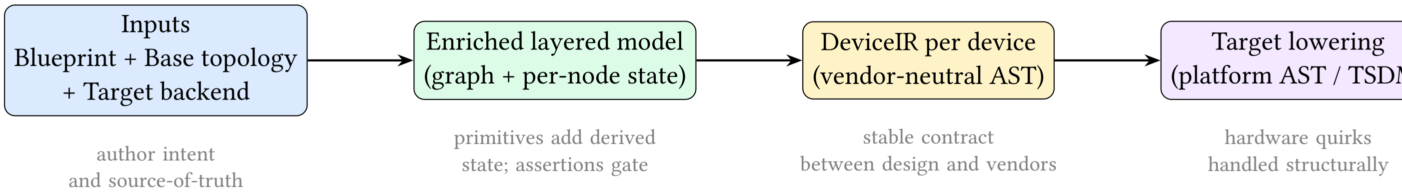

Figure 1. Conceptual compilation funnel. The operator authoring surface is the blueprint; the compiler's core product is an enriched layered graph; the backend lowers that model into vendor-neutral and then target-specific ASTs before serialising final configuration.

Table 1. Stage outputs (conceptual artefacts). These conceptual stages map to the five pipeline layers in §6: Compose = Layer 1, Lower = Layer 3, Adapt = Layer 4, Serialise = Layer 5.

Stage	Output artefact	What it represents
Input	Base topology + blueprint + backend package	Source-of-truth plus a declarative plan plus a reusable target backend.
Compose	Enriched layered model	A single graph containing physical and logical layers, with per-node derived state.
Lower	DeviceIR	Vendor-neutral, structured representation of per-device intent.
Adapt	Target AST / TSDM	Platform-specific structure (interface naming, feature support, syntax constraints).
Serialise	Text configuration files	Deterministic device configs suitable for diff, review, and deployment.

2.2 The network model as a layered graph

NetCfg represents the network as a graph of *nodes* (devices, protocol instances) and *edges* (links, peering sessions). The graph is organised into *layers*: the *physical layer* contains routers, switches, and cables; each *protocol layer* (BGP, OSPF, etc.) contains a parallel set of nodes—one per device participating in that protocol—linked back to their physical origin.

What a "layer" means. A layer is a named, self-contained subgraph representing one plane of intent. The physical layer captures what exists as hardware and cabling. Protocol layers capture logical relationships (adjacencies, sessions, overlay bindings) that are *hosted* on those physical devices. Layers are not "views" over the same nodes; each layer has its own node set. The compiler can therefore represent, in one model, both "device A is cabled to device B" (physical) and "A peers with C via a route-reflector" (BGP) even when those relationships do not coincide.

A physical router **A** thus has a corresponding BGP node **A'** and (optionally) an OSPF node **A"**. Each protocol node carries a *parent mapping* that traces it back to the physical device that hosts it. This cross-layer traceability is how IP addresses allocated on the physical layer flow into BGP configuration without explicit wiring.

Parent mapping and traceability. Every non-physical node stores the identifier of its host physical device (`_source_id` in `data_json`). This forms a many-to-one mapping from overlay nodes to physical nodes: multiple protocol instances (BGP, OSPF, EVPN, IPsec) may all point back to the same physical router. The parent mapping is the only cross-layer link.

Insight: Because overlays never directly reference other layers, the compiler can clone, transform, and

Figure 2 illustrates a minimal example. The physical layer shows three interconnected devices. The protocol layer above contains corresponding protocol nodes; dashed arrows show the parent mapping.

Figure 3 extends the same idea to multiple overlays. Each physical device can host many protocol-layer nodes, one per protocol.

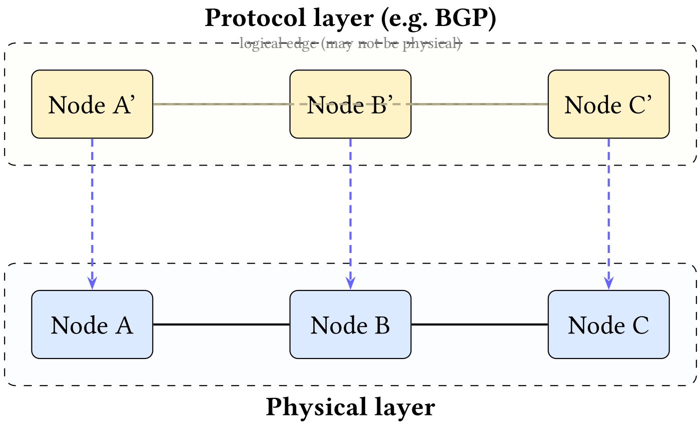

Figure 2. The layered graph model. Physical devices (bottom) have corresponding protocol nodes (top). Dashed arrows show the parent mapping (`_source_id`) that traces each protocol node back to its physical origin. Protocol-layer edges represent logical intent relationships and may be derived from, but need not mirror, physical links.

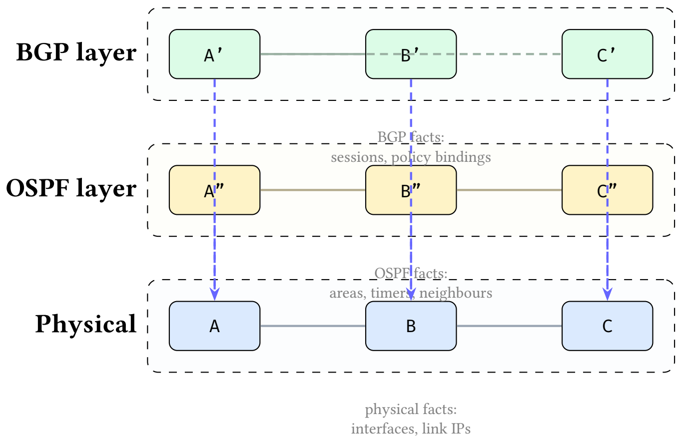

Figure 3. Layered stack as bands. Each column is a physical device (A, B, C). Each protocol layer adds one node per participating device, linked back to the physical node via the parent mapping (vertical dashed arrows). Overlay edges encode protocol-specific logical relationships and may differ across layers.

Where facts live (and how they flow across layers). As a rule of thumb, facts originate in the layer where they are naturally defined: (1) *link and interface facts* (peer interfaces, link subnets, L2/L3 addressing) originate on the physical device nodes; (2) *protocol intent* (neighbour sets, areas, timers, policy bindings) originates on the protocol-layer nodes.

When a protocol node needs a physical fact (e.g., a neighbour IP), it obtains it by following its parent mapping to the host physical node and reading the already-derived fields there. This is how "allocate on physical, consume in overlay" works without bespoke wiring.

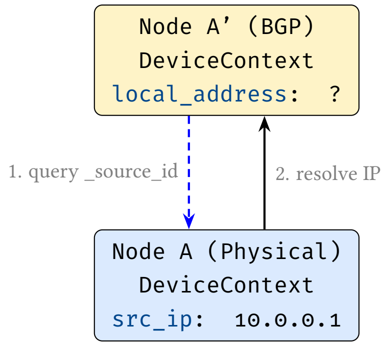

Figure 4. Cross-layer state resolution. Protocol overlays do not maintain redundant state. They resolve physical facts (like allocated IPs) on-the-fly by querying their host physical node via the `_source_id` parent mapping.

Edges within a layer represent direct relationships: physical cables, peering sessions, or policy bindings. Edges *never* cross layers; the parent mapping is the sole mechanism for cross-layer traceability. All mutable state—IP addresses, protocol parameters, security zones—lives on *nodes*, not on edges. Edges exist solely to express relationships between nodes.

Why node-centric state matters. Configuration is ultimately rendered per device. By ensuring every piece of renderable information is attached to nodes (and by keeping edges as pure relationships), the mapping stage can generate a device's configuration by reading exactly one node's accumulated `DeviceContext`. Edges still drive computation (e.g., "allocate a /30 per link"), but the results of that computation are always written back onto the endpoint nodes.

Layering as a compilation mechanism. Layers are created and populated by primitives. For example: `mesh_nodes` adds physical edges; `provision_ips` annotates the physical nodes with link addresses; `build_protocol_layer` clones a selected set of physical nodes into a protocol layer and deep-merges a protocol overlay into the clones. Subsequent primitives can then select either physical nodes or protocol nodes by `layer==...`.

> **Note**
>
> **Design note: protocol edges are logical, not physical.** Protocol-layer edges represent intent-level relationships (e.g., "BGP session exists between A' and C" or "policy binds to this adjacency"). They may be derived from physical links, but they are not required to mirror the physical layer's connectivity.

Kinds of layers (a practical taxonomy)

Layers are intentionally lightweight: they let the model hold multiple simultaneous views of the same network without conflating their semantics. In practice, most NetCfg blueprints use a small, repeatable taxonomy:

1. **Physical layer.** Nodes represent devices; edges represent physical links. Typical state: interface naming, link addressing, management reachability, hardware inventory.
2. **Protocol layers (routing/switching overlays).** Nodes represent a device's participation in a protocol (one node per device per protocol); edges represent logical adjacencies or sessions. Typical state: neighbours, timers, areas/ASNs, policy attachments, protocol feature flags.
3. **Policy and service layers (optional).** Some organisations model policy and services as dedicated layers when the relationships are graph-shaped (e.g., security-zone bindings, NAT relationships,

telemetry subscriptions). Others keep these as node metadata attached to the physical or protocol nodes. The design rule is the same either way: *renderable state stays on nodes; edges express relationships.*

This taxonomy is not prescriptive: the point is that layering preserves separation of concerns while keeping every derived fact traceable back to the physical topology via the parent mapping.

Design rules that build overlays

Layering alone provides representation; *design rules* provide synthesis. In NetCfg, design rules are implemented as primitives that read from the current model (often the physical layer) and then materialise overlay structure and per-device stanzas. They are the bridge from plan topology to protocol overlays.

Two common classes of design rule are:

1. **Overlay construction rules.** `build_protocol_layer` clones selected physical nodes into a protocol layer, records the parent mapping (`_source_id`), and deep-merges protocol defaults into the clones. With `clone_underlying: true`, it also seeds the overlay with a derived copy of the physical connectivity.
2. **Session synthesis rules.** Protocol session primitives (e.g. `build_bgp_session`, `build_ospf_session`, `build_isis_session`)[1] consume the overlay graph and emit configuration stanzas on the participating nodes. These rules are where operator intent becomes concrete neighbours, interfaces, and per-peer parameters.

2.3 Three input documents

Insight: *The operator does not "mesh in the BGP layer" manually. They declare intent; the design rules materialise the overlay relationships and the resulting per-device stanzas in the appropriate layer.*

An operator supplies three documents to the compiler:

1. **Blueprint** — *what* the network should contain. A YAML document declaring layers of primitives that compose the network model: populations of devices, their interconnections, addressing, and protocol state.
2. **Target backend** — *how* to turn the enriched model into per-device configuration. In practice this is a small directory containing (i) target-specific transforms that lower vendor-neutral intent into platform-specific structure, and (ii) templates that serialise that structure into concrete CLI or configuration syntax. Most organisations author this once per platform and reuse it across sites.
3. **Base topology** — the starting graph. A JSON or YAML file listing the initial set of devices (nodes) and links (edges) that the blueprint will enrich.

[1] Session synthesis primitives are planned but not yet in the catalogue; the current workflow uses `build_protocol_layer` with a config overlay to achieve the same result.

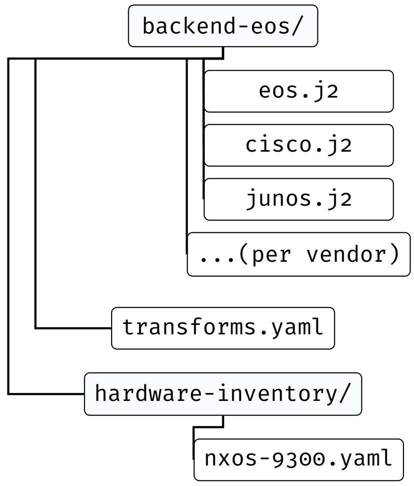

Figure 5. Target backend package structure. Templates render per-vendor syntax; `transforms.yaml` declares TSDM lowering rules; `hardware-inventory/` maps chassis models to slot layouts. Most organisations author this once per platform and reuse it across sites.

The blueprint is the primary authoring surface. An operator writes blueprints; the mapping document is typically authored once per vendor platform and reused across sites. The base topology may come from a network inventory system or be hand-authored for greenfield designs.

2.4 Blueprints compose the model declaratively

A blueprint organises primitives into ordered *layers*. Each layer declares a set of primitives; each primitive selects a subset of the graph and enriches the nodes it touches. The result is a progressively richer model, as Figure 6 illustrates.

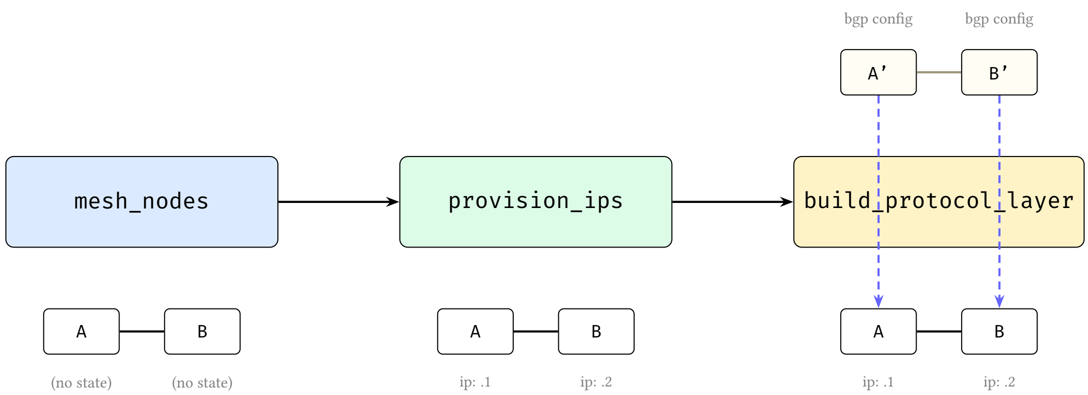

Figure 6. Progressive model enrichment. Left: `mesh_nodes` establishes edges between bare nodes. Centre: `provision_ips` annotates nodes with IP addresses. Right: `build_protocol_layer` clones nodes into a protocol layer and deep-merges a protocol overlay into the clones; dashed arrows show the parent mapping back to the physical devices. At each stage the model grows richer; no primitive needs to know what others have done.

Three properties make this composition model work:

1. **State lives on nodes, not edges.** Edges define relationships—physical links, peering sessions, policy bindings. All mutable, renderable state is written to the node at each end of an edge. Edges may carry static relationship metadata (e.g., labels), but do not carry configuration-bearing state.
2. **Primitives compose, not override.** Each primitive enriches the model additively. No primitive needs to know what other primitives have done; the model accumulates state layer by layer.

3. **Primitives are typed model transforms.** Each primitive produces a specific kind of enrichment: new graph structure (overlay nodes and edges), allocated identifiers (IP addresses, ASNs, VLANs), compiled policy objects (firewall rules, route-maps), or protocol stanzas (VXLAN VNIs, EVPN route-targets). Primitives never name specific devices; they reference *groups* and *selectors* (graph queries that resolve against the current model state), so the same blueprint can operate on topologies of different sizes.

2.5 Mini worked example (conceptual)

The following tiny blueprint fragment illustrates the layered-model flow without requiring any knowledge of Rust, template engines, or selector internals.

Listing 1. A minimal blueprint fragment (layered model + design rules).

```yaml
layers:
  - name: physical
    primitives:
      - type: mesh_nodes
        selector: "nodes[role=='core']"
        mesh_type: full

      - type: provision_ips
        selector: "edges[true]"
        pool: 10.0.0.0/24
        subnet_size: 30

  - name: bgp
    requires: [physical]
    primitives:
      - type: build_protocol_layer
        selector: "nodes[layer=='physical' && role=='core']"
        layer: bgp
        clone_underlying: true
        config: { bgp: { enabled: true } }

      # Protocol sessions are derived by design rules from the overlay graph.
      # The overlay edges created here are intent-level relationships, not
      # necessarily identical to physical links.
      - type: build_bgp_session
        selector: "edges[true]"

assertions:
  - name: core-has-link-address
    severity: error
    select: "nodes[role=='core']"
    check: { type: field_exists, field: src_ip }
```

Conceptually, this executes as follows:

1. **Start with a base topology** containing core routers as physical nodes.
2. **`mesh_nodes`** creates physical edges between those nodes.
3. **`provision_ips`** examines the edges to decide which links need subnets, then writes the resulting link addresses onto the endpoint *nodes* (not onto the edges).
4. **`build_protocol_layer`** clones the physical core nodes into a BGP overlay layer and records a parent mapping (`_source_id`) back to the physical devices. The protocol overlay config is deep-merged into the clones.

5. **Overlay edges encode protocol intent.** With `clone_underlying: true`, the BGP overlay begins with derived connectivity copied from the physical layer. A session design rule such as `build_bgp_session` then reads that overlay graph and writes per-device neighbour stanzas onto the participating nodes.
6. **Assertions** verify the model is well-formed (here: every core device has a link address) before any vendor-specific configuration is produced.

The key point is that the operator never names a specific interface or assigns an IP address directly. The blueprint describes intent, and the compiler derives concrete per-device state from the layered topology relationships.

2.6 The select–enrich–validate pattern

Every primitive follows the same three-phase pattern (Figure 7):

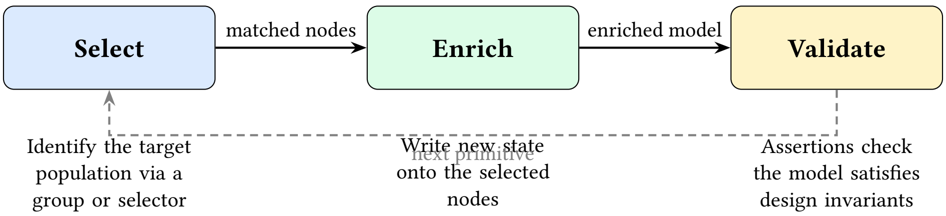

Figure 7. The select–enrich–validate cycle. Every primitive selects a population of nodes or edges and enriches them with new state. Validation is expressed as assertions over the model and may run as an explicit checkpoint, a layer barrier, or a final gate before rendering.

- **Select.** A group reference or CEL selector query identifies which nodes (or edges) the primitive will operate on. The selector is evaluated against the current graph state, so earlier primitives' enrichments are visible to later selectors.
- **Enrich.** The primitive performs a typed model mutation on the selected population. Depending on the primitive's tier, this may produce new graph structure (overlay nodes and edges), allocated identifiers (IP addresses, ASNs, VLAN IDs), compiled policy objects (firewall rules, route-maps, prefix-lists), or protocol stanzas (VXLAN VNIs, EVPN route-targets, BGP peering parameters). State always accumulates; primitives never delete or overwrite each other's contributions.
- **Validate.** Assertions verify that the model satisfies the operator's invariants (e.g., "every node has a link address" or "no two links share a subnet"). Assertions may run immediately (as checkpoints), at the end of a layer, and again as a final gate before any configuration is generated.

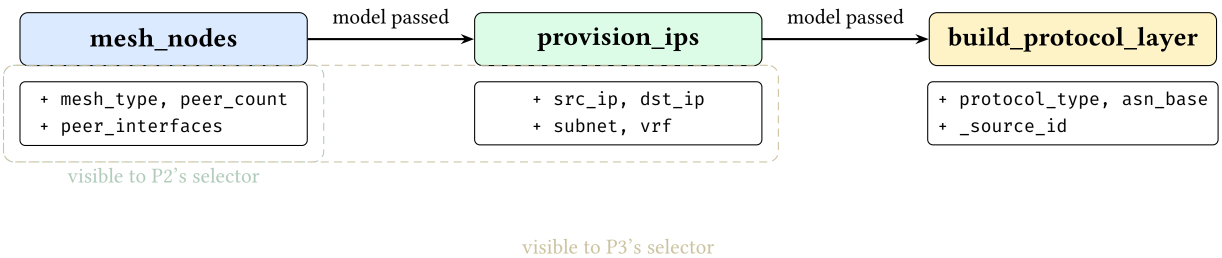

Figure 8. Selector visibility timeline. Each primitive's selector evaluates against the *current* graph state, which includes all fields written by earlier primitives. State accumulates monotonically.

Validation checkpoints. NetCfg supports three complementary validation styles, all expressed as assertions over the current model state: (1) *mid-pipeline checkpoints* (via `assert_state`) that fail fast after a critical enrichment step; (2) *layer barriers* that run after a layer's primitives execute; and (3) *final* *gates* that run before any configuration is rendered. Conceptually, all three ask the same question—"does the model satisfy the design invariants?"—but they fire at different times for faster feedback.

This uniform pattern means that learning one primitive teaches the operator the shape of every primitive: declare a selector, declare the desired state, and optionally assert invariants. The formal definitions and concrete DSL syntax follow in §3 and §5.

3 Background: Network Configuration as Code

Concept first. The purpose of the formal model in this section is to make the compiler's behaviour precise: what constitutes a node, what constitutes an edge, how layers coexist, and how per-device state accumulates. Implementation choices (Rust types, query engines, evaluation languages) are replaceable details; the stable ideas are: a layered graph, node-centric state, and deterministic derivation of device configuration from topology relationships.

NetCfg models the network as a directed graph $G = (V, E, \tau, \lambda, D)$ where V is a set of nodes (devices, protocol instances), $E \subseteq V \times V$ is a set of edges (links, peering sessions), $\tau : V \to T$ assigns each node a *type* (Router, Switch), $\lambda : V \to L$ assigns each node a *layer*, and $D : T \to DataFrame$ maps each type to a columnar property store (Polars DataFrame).

3.1 Layers

A layer represents a plane of the network: `physical` for the hardware topology, `bgp` for BGP peering sessions, `ospf` for OSPF adjacencies. A single physical router (node 1, layer `physical`) may have corresponding protocol nodes (node 101, layer `bgp`; node 201, layer `ospf`), each linked to node 1 via a *parent mapping* (`_source_id` in the node's `data_json`).

Layers allow the compiler to model the physical and logical topologies simultaneously, with cross-layer references preserving traceability from a protocol node back to the physical device that hosts it.

3.2 Per-node state: DeviceContext

How per-node state relates to the layered model. The layered graph defines *where* intent lives: a device's physical node captures hardware-adjacent facts (interfaces, link addressing, management), while each protocol-layer node captures logical intent for that protocol (BGP neighbours, OSPF areas, EVPN VNIs). In both cases, the compiler stores derived configuration-bearing state *on the node* so that rendering is a local operation: to generate configuration for a device in a given layer, the backend reads exactly one node's accumulated state.

Conceptually: a node's "accumulated facts". Think of `DeviceContext` as a typed view over a node's accumulated facts at that point in the pipeline. Primitives add facts ("this interface has 10.0.0.1/30", "this device is in VRF X", "these are my BGP peers"), and assertions check that required facts are present before lowering continues.

Each node's properties are stored as a JSON blob (`data_json`) in a Polars DataFrame keyed by node type. While JSON is unstructured, Polars is used for ultra-fast topological graph queries and edge traversals, unpacking the JSON payload on-demand only when evaluating CEL [12] expressions. NetCfg deserialises this blob into a typed `DeviceContext` struct:

Listing 2. DeviceContext: the per-node state model.

```
pub struct DeviceContext {
    pub hostname: String,
    pub device_os: Option<String>,        // template selector
    pub management_ip: Option<String>,
    pub src_ip: Option<String>,           // written by provision_ips
```

```
    pub dst_ip: Option<String>,           // written by provision_ips
    pub subnet: Option<String>,           // written by provision_ips
    pub src_ipv6: Option<String>,         // dual-stack support
    pub dst_ipv6: Option<String>,
    pub subnet_v6: Option<String>,
    pub vrf: Option<String>,              // VRF domain
    pub peer_interfaces: Option<HashMap<String, String>>,
    pub next_interface_index: Option<u32>,
    pub stanzas: Vec<Stanza>,             // accumulated config units
    pub metadata: HashMap<String, JsonValue>, // extension point
}
```

Primitives progressively enrich this context: `mesh_nodes` establishes edges and writes `mesh_type`/`peer_count` into metadata; `provision_ips` writes IP fields; `build_protocol_layer` deep-merges config overlays into the cloned node's metadata. By the time rendering occurs, each node's `DeviceContext` contains all information needed to produce its configuration file.

> **Note**
>
> The `metadata` field uses `serde(flatten)`, so any JSON key not matching a named field is captured here. This makes `DeviceContext` extensible without schema changes.

> **Note**
>
> **Graph engine dependency.** The underlying graph is implemented by the NTE topology engine [13], which provides a petgraph-backed directed graph with Polars DataFrames for columnar property storage. NetCfg depends on `nte_topology` for the graph and `ank_nte` for the mapping evaluator and DeviceIR types.

4 Intent and Policy Expression

A blueprint is a *plan document*: a declaration of how the network model should be composed. The operator never manipulates nodes, edges, or IP addresses directly. Instead, they declare *what* the model should contain—populations of devices, the relationships between them, the addresses and protocol state those devices need—and the compiler builds the model through successive primitive applications.

This section frames the conceptual model that operators use when writing blueprints, before §5 catalogues the concrete syntax.

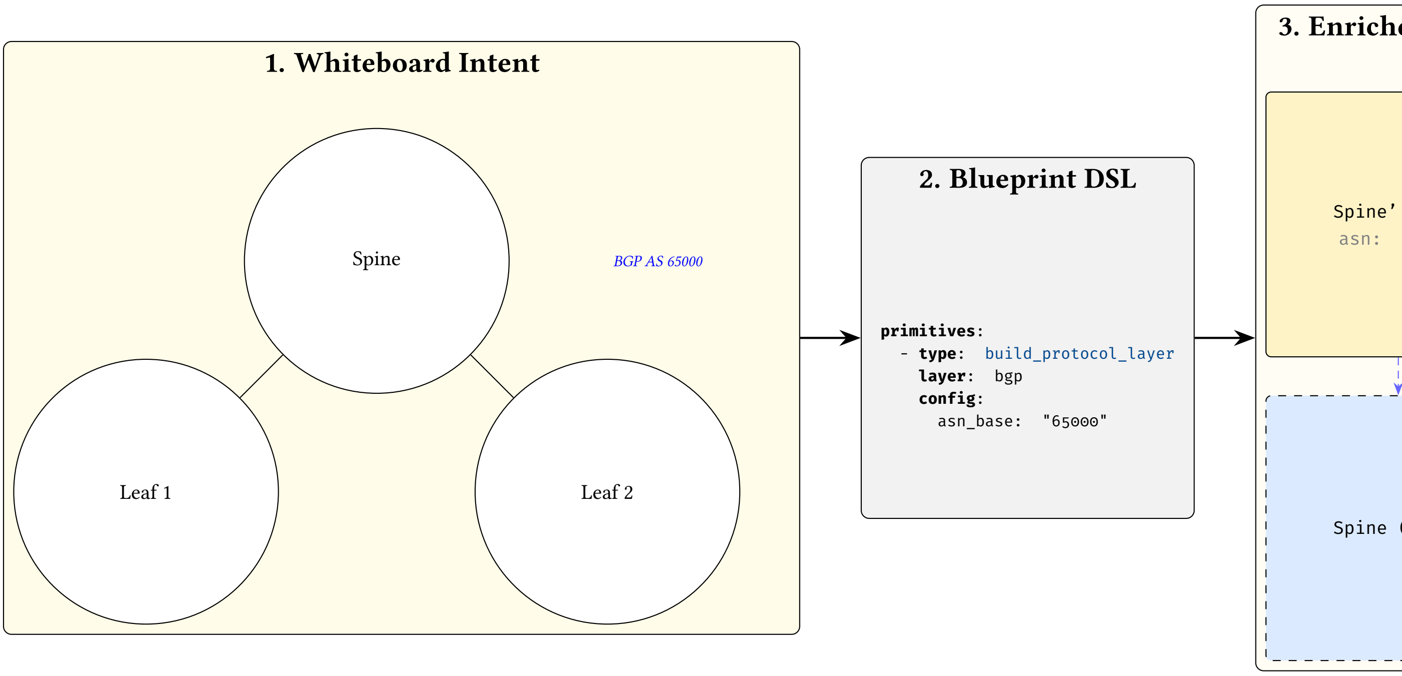

Figure 9. The Whiteboard-to-Plan (W2P) strategy. An architect's visual intent (left) is formalised into declarative YAML primitives (centre), which the compiler executes to produce an enriched, layered graph representation (right).

4.1 The five concerns of a network blueprint

Every blueprint addresses five orthogonal concerns. Table 2 shows how each maps to a DSL construct and the design question it answers.

Table 2. The five concerns of a network blueprint.

Concern	DSL construct	Question answered
Population naming	`groups:`	Which devices belong to which roles?
Topology intent	`layers:` / primitives	How should they be connected and what state should they carry?
Policy constraints	routing/access/prefix-list primitives	What traffic rules should be enforced?
Design invariants	`assertions:` + `rules:`	What properties must always hold?
Change safety	`blast_radius:`	How much change is acceptable per commit?

The key insight is that every concern operates on the *network model* —the layered graph described in §3. The operator declares populations, intent, policy, and invariants; the compiler maps these into graph mutations that progressively enrich the model's nodes.

4.2 From plan to network model: declarative composition

§2.4 introduced the five primitive tiers and their mutation semantics. This subsection shows how those tiers interact at the implementation level.

`provision_ips` iterates edges to determine which node pairs need addresses, but writes `src_ip` and `dst_ip` onto the respective *nodes*—not onto the edge itself. `build_protocol_layer` clones selected nodes into a new graph layer and merges protocol configuration into them via deep merge. Each primitive sees the enriched model left by its predecessors, but never needs to know what they did.

: Node-Centric State

All per-device state is written to nodes via `DeviceContext`. Edges carry no mutable state; they exist solely to express relationships between nodes. This invariant simplifies the mapping stage: to produce a device's configuration, the mapping evaluator reads exactly one node.

Groups and selectors are the glue that lets these transforms find their targets. Groups let the operator name populations once (e.g. `$core_routers`, `$edge_links`, `$management_nodes`) and reference them in every primitive and assertion. Selectors compile group references and attribute predicates into CEL graph queries that the compiler evaluates against the live topology.

4.3 Expressing policy at the plan layer

Policy primitives follow the same declarative pattern: select a population, declare the desired policy state, and let the compiler write it to the model. The operator never constructs individual ACL entries or route-map clauses. Instead, they declare the *intent*—which traffic classes to permit or deny, which prefixes to advertise, which communities to match—and the primitive translates this into the appropriate policy objects on each selected node.

The separation between *topology intent* (layers 1–2: structure and addressing) and *policy intent* (layer 3+: traffic rules) is a deliberate design choice. Topology primitives build the graph; policy primitives annotate nodes with forwarding constraints. Both operate through the same select-and-enrich mechanism, but they address orthogonal concerns and can be developed, tested, and imported independently.

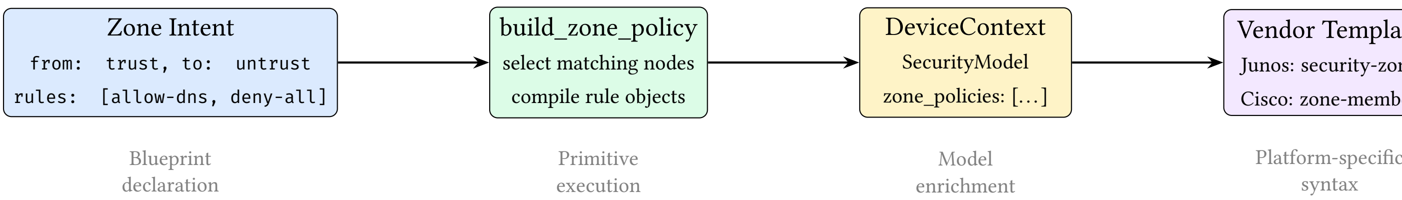

Figure 10. Policy compilation flow. The operator declares zone intent; the `build_zone_policy` primitive compiles it into typed rule objects on `DeviceContext`; vendor templates render platform-specific syntax.

Case study: multi-tenant isolation

The multi-tenant data-centre case study (`examples/case_studies/04_multi_tenant_dc.yaml`) illustrates how these concerns compose. The operator declares: (1) groups that name device roles and per-tenant link populations; (2) a hub-and-spoke mesh for the fabric; (3) per-VRF address pools; (4) access policies that deny cross-VRF traffic; (5) assertions that every device has a tenant assignment and that each tenant's subgraph is connected; and (6) a blast-radius limit. At no point does the operator name a specific device or IP address—the compiler derives all concrete state from the declared populations and intent.

4.4 Reusable design-rule libraries

Complex assertions can be factored into named `rules:` (CEL macros) and shared across blueprints via `imports:`. Two standard rule libraries and a protocol fragment library ship with NetCfg:

- **`datacenter-rules.yaml`** — a portable linting ruleset for data-centre fabrics. It validates OSPF point-to-point subnet lengths, MTU consistency across edges, BGP private-ASN ranges, BGP neighbour IP existence, IS-IS level consistency, and IS-IS NET address formatting. Blueprints import it with a single line: `imports: ['datacenter-rules.yaml']`.

- **`hardware-lowering.yaml`** — vendor-specific AST transforms for Cisco NX-OS, Juniper Junos, Cisco IOS-XR, and VyOS. It remaps abstract `Ethernet` interface names into platform-specific formats (e.g. `Ethernet0` → `ge-0/0/0` on Junos) and standardises overlay MTU to 9216.
- **`protocols/*.yaml`** — a library of 31 importable protocol fragments, each providing a single `build_protocol_layer` invocation with sensible defaults. Covers routing (BGP, OSPF v2/v3, IS-IS, RIP, LDP, RSVP-TE, SR-MPLS), switching (VXLAN, EVPN, STP/RSTP, LACP, MLAG), security (IPsec/IKEv2, WireGuard, MACsec, 802.1X, RADIUS, TACACS+), services (NTP, Syslog, SNMP, DHCP Relay), multicast (PIM-SM, IGMP), telemetry (NetFlow/IPFIX/sFlow), and layer 2 (LLDP, ARP, BFD, VRRP, GRE). Blueprints import individual protocols: `imports: ['protocols/ospf.yaml', 'protocols/bfd.yaml']`.

Ten case studies in `examples/case_studies/` demonstrate production-scale blueprints: ISP dual-stack peering, campus compliance audit, WAN OSPF→IS-IS migration, multi-tenant DC with per-VRF isolation, SP MPLS core with SR-MPLS, campus 802.1X NAC, SD-WAN with IPsec overlay, multicast video distribution, zero-trust DC microsegmentation, and PCI-DSS financial compliance.

Assertions reference named rules via the `use_rule` check type, keeping the assertion block concise:

Listing 3. Referencing named rules via `use_rule`.

```yaml
rules:
  link-addressed: "has(node.src_ip) && node.src_ip != ''"
  protocol-bound: "has(node.metadata.asn) && node.metadata.asn != ''"

assertions:
  - name: every-edge-device-has-address
    severity: error
    select: nodes[role=='edge']
    check:
      type: use_rule
      name: link-addressed
```

5 Blueprint DSL: Declarative Topology Transformations

A blueprint is a YAML document declaring what topology transformations to apply. It is the "what" of the network design.

5.1 Schema and layer ordering

Listing 4. Blueprint top-level schema (Rust type definition). Appendix A provides the equivalent YAML reference.

```rust
pub struct Blueprint {
    pub version: u32,                  // must be >= 1
    pub imports: Vec<String>,          // relative paths to merge
    pub layers: Vec<LayerSpec>,        // ordered layer blocks
    pub assertions: Vec<AssertionSpec>, // post-execution checks
    pub rules: HashMap<String, String>, // named CEL macros
    pub groups: HashMap<String, GroupSpec>, // named node sets
    pub blast_radius: Vec<BlastRadiusSpec>, // change policies
    pub transforms: Vec<TransformationSpec>, // vendor adaptation
    pub hardware_library: HardwareLibrary,  // chassis/linecard defs
}
```

```rust
pub struct LayerSpec {
    pub name: String,             // non-empty
    pub requires: Vec<String>,    // must name earlier layers
    pub primitives: Vec<Primitive>,
}
```

Three validation passes run at parse time:

1. **Schema validation** via `serde_valid`: field types, non-empty names, integer ranges. Unknown YAML keys are rejected via `deny_unknown_fields`, catching typos at parse time.
2. **CEL selector compilation**: each selector string is compiled into an evaluable CEL program, surfacing syntax errors before execution.
3. **Layer ordering validation**: a forward scan ensures every `requires` entry names a previously-declared layer. This catches ordering bugs at parse time.

The optional `imports` field lists relative file paths to other blueprint files. At parse time, layers and assertions from imported files are merged into the importing blueprint, enabling reusable assertion libraries (e.g. a standard linting ruleset shared across projects).

> **Warning**
>
> Duplicate layer names are permitted. Multiple `LayerSpec` blocks with `name:  input` are common in practice—each block adds primitives to the same layer. The `requires` mechanism enforces inter-layer ordering, not intra-layer uniqueness.

5.2 Selector DSL (CEL surface syntax)

Selectors identify which nodes or edges a primitive operates on. Conceptually, a selector is simply a boolean query over the current model: "for each node (or edge), should this primitive apply here?" The concrete surface syntax is a lightweight wrapper around Google's Common Expression Language (CEL):

Listing 5. Selector syntax examples.

```yaml
# Select all nodes
selector: "all()"

# Select nodes by property (e.g. region, role, type)
selector: "nodes[region=='eu-west']"
selector: "nodes[role=='core' && layer=='input']"
selector: "nodes[type=='Router' and region=='us-east']"

# Select edges
selector: "edges[src>=0]"
selector: "edges[true]"
```

The selector parser:

1. Identifies whether the selector targets `nodes` or `edges` and extracts the bracketed predicate.
2. Normalises minor syntax differences (e.g. `and` vs `&&`) so the predicate is unambiguous.
3. Compiles the predicate into an evaluable program.

At evaluation time, the selector ranges over all nodes (or edges) in the topology and returns the subset for which the predicate is true. This means selectors are always evaluated against the *current* model state: earlier primitives can add fields that later selectors use.

Mechanically, the compiler binds each entity's properties as variables (`id`, `type`, `layer`, plus all `data_json` fields, flattened and accessible via dot notation) and evaluates the compiled predicate.

> **Note**
>
> **Why missing fields are not errors.** Because the model is layered, not every node has every field: a physical node will have interface and addressing facts, while a protocol node will have protocol-specific facts. Selectors must therefore tolerate heterogeneous populations. Mechanically, undeclared variable references (e.g., querying a field that some nodes lack) silently evaluate to `false` rather than producing an error. This lets a single selector range over mixed layers while still precisely matching only the nodes that carry the relevant facts.

For IP-valued fields, the selector also binds a `{field}_len` variable containing the prefix length, enabling selectors like `nodes[subnet_len == 30]`.

> **Note**
>
> **Layer-aware selector patterns.** Because the model is layered, robust selectors typically constrain both *what* a node is (role/type) and *where* it lives (layer). Common patterns include:
>
> ```
> # Physical devices only
> selector: "nodes[layer=='physical' && role=='core']"
>
> # Protocol overlay nodes only (e.g. BGP layer)
> selector: "nodes[layer=='bgp' && role=='core']"
>
> # Edges within a layer (physical links, protocol sessions)
> selector: "edges[layer=='physical']"
>
> # Guard against heterogeneous fields (match only nodes that have the facts)
> selector: "nodes[layer=='bgp' && has(node.bgp) && node.bgp.enabled==true]"
> ```

5.3 Named groups

The `groups:` section defines reusable named node or edge sets. Group names are prefixed with $ when referenced in selectors. Two forms are supported:

Listing 6. Group definitions: shorthand and full form.

```
groups:
  # Shorthand: just a selector string (e.g. by role)
  core_routers: "nodes[role=='core']"
  access_switches: "nodes[role=='access']"

  # Full form with kind and kind-specific fields
  dmz:
    selector: "nodes[zone=='dmz']"
    kind: security_zone
    trust_level: 10
```

Groups are resolved before primitives execute. The selector parser expands `$group_name` references inline, and cycle detection rejects circular group dependencies at parse time. When blueprints import other files, groups from imported files are merged into the importing blueprint's namespace (name collisions are last-writer-wins).

The union operator (`|`) combines groups:

```
  all_devices: $core_routers | $access_switches
```

This creates a set containing all nodes matching either group.

When a group has `kind: security_zone`, an optional `interface_selector` field may be supplied to express interface-level zone membership. This selector targets edges rather than nodes, enabling fine-grained policies where different interfaces on the same device belong to different zones:

```yaml
dmz:
  selector: "nodes[zone=='dmz']"
  kind: security_zone
  trust_level: 10
  interface_selector: "edges[intf_role=='dmz']"
```

5.4 Named objects

The top-level `objects:` section defines reusable address and service objects. These are referenced in zone policy and NAT policy rules using the $ prefix (e.g. `$rfc1918`, `$web_services`). Both sub-maps default to empty, so omitting `objects:` is backward-compatible.

Listing 7. Named address and service objects.

```yaml
objects:
  addresses:
    rfc1918:
      prefixes:
        - "10.0.0.0/8"
        - "172.16.0.0/12"
        - "192.168.0.0/16"
    corp_v6:
      prefixes_v6:
        - "2001:db8::/32"
  services:
    web_services:
      entries:
        - { protocol: tcp, port: 80 }
        - { protocol: tcp, port: 443 }
    dns:
      entries:
        - { protocol: udp, port: 53 }
        - { protocol: tcp, port: 53 }
```

`AddressObject` supports dual-stack addressing via separate `prefixes` (IPv4) and `prefixes_v6` (IPv6) fields. Prefixes are stored as strings and validated to `ipnet::IpNet` at object registry build time.

`ServiceObject` contains one or more `ServiceEntry` pairs of `protocol` and `port`. In zone policy or NAT rules, use `service: $web_services` in the `match:` block to reference a named service object.

5.5 Primitive catalogue

NetCfg provides twenty-two primitive types organised into five tiers, each distinguished by what it *mutates* in the network model.[2]

[2]The `wasm_plugin` primitive requires the optional `wasm` Cargo feature; a default build exposes twenty-one primitives.

Note: primitive naming

The primitive types documented below use their stable DSL aliases (e.g. `allocate_resources`, `build_routing_policy`). The Rust source uses canonical names (e.g. `provision_resources`, `policy_routing`) with the DSL names accepted as `serde` aliases for backward compatibility. Both names are accepted in blueprint YAML; the canonical name appears in Rust tracebacks and error messages. Table 3 lists all renames.

Table 3. Primitive canonical names and DSL aliases.

Canonical (Rust)	DSL alias	Tier
`provision_resources`	`allocate_resources`	Allocation
`define_ip_pool`	`global_pool`	Allocation
`define_resource_pool`	`global_resource_pool`	Allocation
`policy_routing`	`build_routing_policy`	Policy
`policy_access`	`build_access_policy`	Policy
`policy_prefix_list`	`build_prefix_list`	Policy
`policy_community_list`	`build_community_list`	Policy
`policy_bgp_safety`	`generate_safe_bgp_filters`	Policy
`overlay_vxlan`	`build_vxlan`	Overlay
`overlay_evpn`	`build_evpn`	Overlay

The five tiers are:

- **Topology** — create new nodes and edges (graph structure).
- **Allocation** — assign derived identifiers to existing nodes (IP addresses, ASNs, VLANs).
- **Policy** — compile high-level declarations into per-device rule sets (zone policies, ACLs, route-maps).
- **Overlay** — attach protocol-specific configuration stanzas (VXLAN, EVPN, BGP peering).
- **Meta** — annotate, gate, or extend the pipeline without producing device configuration.

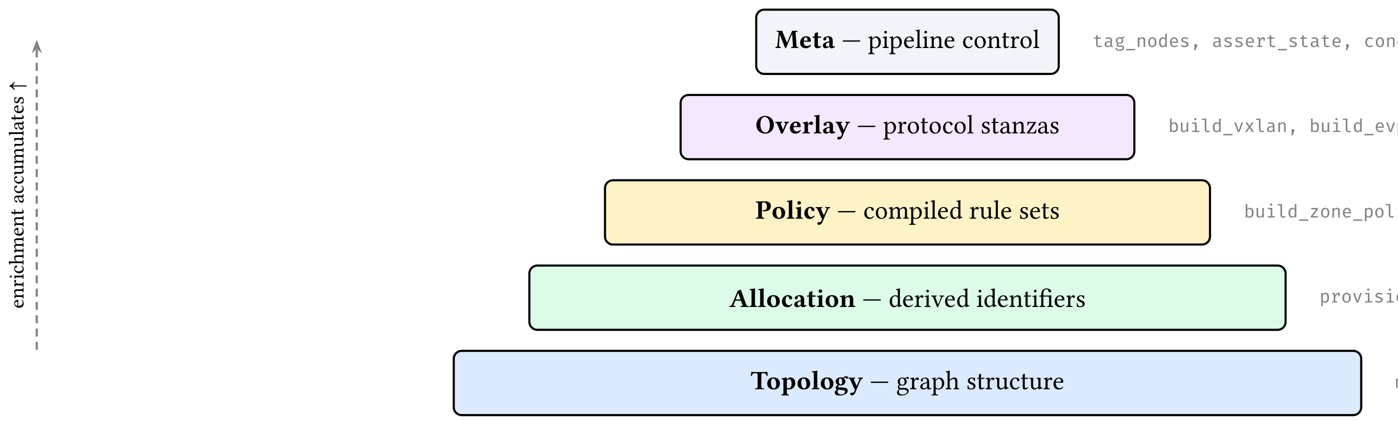

Figure 11. Primitive tier hierarchy. Lower tiers create structure; upper tiers annotate and configure existing nodes. Each tier builds on state written by earlier tiers.

All primitives follow the *select* → *enrich* → *validate* pattern: select a target population via graph queries, compute the enrichment, then write results to `DeviceContext` or the graph structure.

Table 4 summarises the catalogue. Full field tables follow.

Table 4. Primitive catalogue overview. Status: **S**table, **B**eta, **E**xperimental.

Primitive	**Tier**	**Purpose**	**Status**
`mesh_nodes`	Topology	Create edges between selected nodes (full, hub-and-spoke)	S
`build_protocol_layer`	Topology	Clone nodes into a named protocol overlay layer	S
`provision_ips`	Allocation	Allocate IP subnets to edges from a CIDR pool	S
`allocate_resources`	Allocation	Generic pool allocation for ASNs, VLANs, etc.	S
`global_pool`	Allocation	Register a named IP pool for hierarchical carving	S
`global_resource_pool`	Allocation	Register a named resource pool (non-IP)	S
`build_routing_policy`	Policy	Attach routing policy stanzas (route-maps)	S
`build_access_policy`	Policy	Attach access-control policy stanzas (ACLs)	S
`build_prefix_list`	Policy	Create named prefix-list entries on matched nodes	S
`build_community_list`	Policy	Create named community-list entries on matched nodes	S
`generate_safe_bgp_filters`	Policy	Auto-generate RFC 1918 bogon filters	S
`build_zone_policy`	Policy	Zone-based firewall policies with ordered rules	B
`build_nat_policy`	Policy	NAT policies (SNAT interface/pool, DNAT port-forward)	B
`build_vxlan`	Overlay	Assign VXLAN VNI and multicast group bindings	S
`build_evpn`	Overlay	Assign EVPN route-distinguisher and route-target	S
`tag_nodes`	Meta	Stamp key–value metadata onto matched nodes	S
`map_hardware_inventory`	Meta	Assign chassis model and line-card slot mappings	S
`assert_state`	Meta	Mid-pipeline assertion checkpoint	S
`conditional`	Meta	CEL-based branching within blueprints	S
`custom_primitive`	Meta	Named reusable primitive group with parameters	S
`inject_secrets`	Meta	Read environment variables into node metadata	S
`wasm_plugin`	Meta	Execute a WebAssembly plugin against matched nodes	E

5.5.1 `mesh_nodes`

Creates edges between selected nodes. Supports three mesh topologies: `full` ($\binom{n}{2}$ edges), `hub_and_spoke` (hubs full-meshed, spokes connected to all hubs), and `route_reflector` (same topology as hub-and-spoke). Idempotent: existing edges are detected via a `HashSet<(i32, i32)>` and skipped.

Table 5. `mesh_nodes` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for nodes to mesh
`mesh_type`	String	Yes	`full`, `hub_and_spoke`, or `route_reflector`
`hub_selector`	String	H&S	Selector for hub nodes
`spoke_selector`	String	H&S	Selector for spoke nodes
`edge_properties`	JSON	No	Properties for created edges (deferred; see §12.1)
`naming_strategy`	String	No	Interface naming: `eth`, `cisco_ge`, `junos_ge`

Example: full mesh within a site

```yaml
- type: mesh_nodes
  selector: "nodes[site='a']"
  mesh_type: full
```

With 4 nodes at site A, this creates $\binom{4}{2} = 6$ edges and assigns interface names `eth0`, `eth1`, etc. to each node's `peer_interfaces` map.

5.5.2 `provision_ips`

Allocates subnets from a CIDR pool to edges. See §7.1 for the full five-pass algorithm.

Table 6. `provision_ips` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for edges
`pool`	String	Yes	CIDR (e.g. `10.0.0.0/24`) or global pool name
`subnet_size`	u8	No	Prefix length per edge (default: 30 for IPv4, 126 for IPv6)
`ipv6_pool`	String	No	IPv6 CIDR or global pool name for dual-stack
`ipv6_subnet_size`	u8	No	IPv6 prefix length per edge
`strategy`	String	No	`dense` (default), `sparse`, or `hash`
`pool_size`	u32	No	When referencing a global pool, size of block to carve
`vrf`	String	No	VRF domain (default: `default`)

5.5.3 `build_protocol_layer`

Clones physical nodes into a named protocol layer. See §7.2 for the overlay construction mechanism.

Table 7. `build_protocol_layer` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for source-layer nodes
`layer`	String	Yes	Target layer name (must differ from source)
`config`	JSON	Yes	Config overlay deep-merged into cloned nodes
`clone_underlying`	bool	No	Also clone edges between selected nodes (default: false)

5.5.4 `allocate_resources`

Generic pool allocator for non-IP resources (ASNs, VLANs, loopback IDs).

Table 8. `allocate_resources` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for nodes
`resource_type`	String	Yes	Key name in `DeviceContext.metadata`
`pool`	String	Yes	Range (e.g. `65000-65999`) or global pool name
`strategy`	String	No	`dense` (default)
`pool_size`	u32	No	When referencing a global pool, number of values to carve

5.5.5 `global_pool`

Registers a named IP address pool at blueprint scope, enabling hierarchical pool carving across multiple `provision_ips` primitives.

Table 9. `global_pool` fields.

Field	Type	Required	Description
`name`	String	Yes	Pool identifier referenced by `provision_ips`
`pool`	String	Yes	CIDR range (e.g. `10.0.0.0/8`)
`vrf`	String	No	VRF domain (default: `default`)

5.5.6 `global_resource_pool`

Registers a named non-IP resource pool at blueprint scope.

Table 10. `global_resource_pool` fields.

Field	Type	Required	Description
`name`	String	Yes	Pool identifier
`resource_type`	String	Yes	Resource category (e.g. `asn`)
`pool`	String	Yes	Value range (e.g. `65000-65999`)

5.5.7 `conditional`

CEL-based branching. The `condition` is evaluated as a CEL expression; if true, `then_primitives` execute, otherwise `else_primitives` (if present) execute.

Table 11. `conditional` fields.

Field	Type	Required	Description
`condition`	String	Yes	CEL expression
`then_primitives`	Vec<Primitive>	Yes	Primitives if condition is true
`else_primitives`	Vec<Primitive>	No	Primitives if condition is false

5.5.8 `custom_primitive`

A named, reusable group of primitives with parameters. The `parameters` map is pushed onto a stack and accessible within the nested primitives.

Table 12. `custom_primitive` fields.

Field	Type	Required	Description
`name`	String	Yes	Primitive name (for reporting)
`parameters`	HashMap<String, JsonValue>	Yes	Named parameters
`primitives`	Vec<Primitive>	Yes	Nested primitive list

5.5.9 `inject_secrets`

Reads environment variables into node metadata. Keys in the `secrets` map are metadata field names; values are environment variable names.

Table 13. `inject_secrets` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for target nodes
`secrets`	HashMap<String, String>	Yes	{metadata_key: env_var_name}

5.5.10 `build_routing_policy`

Attaches routing policy stanzas to matched nodes. Each statement defines a route-map entry with match conditions and set actions, stored as a `Stanza` in `DeviceContext.stanzas` for downstream template rendering.

Table 14. `build_routing_policy` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for target nodes
`policy_name`	String	Yes	Policy name (e.g. `BGP-EXPORT`)
`statements`	Vec<RoutingPolicyStatement>	Yes	Ordered route-map entries

Each RoutingPolicyStatement has fields: name (sequence number), action (permit or deny), optional match_condition, and optional set_action. Templates access stanzas via device.stanzas | selectattr('kind', 'equalto', 'routing_policy').

Example: BGP export policy

```yaml
- type: build_routing_policy
  selector: "nodes[role='border']"
  policy_name: "BGP-EXPORT"
  statements:
    - name: "10"
      action: permit
      match_condition: "prefix-list CUSTOMER"
      set_action: "local-preference 100"
    - name: "20"
      action: deny
```

5.5.11 build_access_policy

Attaches access-control policy stanzas to matched nodes. Each rule defines a firewall or ACL entry with source, destination, protocol, and port fields.

Table 15. build_access_policy fields.

Field	Type	Required	Description
selector	String	Yes	CEL selector for target nodes
policy_name	String	Yes	Policy name (e.g. EDGE-ACL)
rules	Vec<AccessPolicyRule>	Yes	Ordered ACL entries

Each AccessPolicyRule has fields: name, action (permit/deny), optional source, destination, protocol, and port. Both policy primitives enforce SelectorTarget::Nodes—policies cannot target edges.

Example: DMZ access control list

```yaml
- type: build_access_policy
  selector: "nodes[zone='dmz']"
  policy_name: "UNTRUSTED-TO-INTERNAL"
  rules:
    - name: "100"
      action: deny
      protocol: tcp
      destination: "10.0.0.0/8"
    - name: "999"
      action: permit
```

5.5.12 wasm_plugin

[Experimental] Executes a WebAssembly plugin binary against matched nodes. Requires the wasm Cargo feature flag (–features wasm); without it, the pipeline returns a descriptive error. Uses the wasmtime [14] v18 runtime.

Table 16. `wasm_plugin` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for target nodes
`plugin_path`	String	Yes	Filesystem path to compiled `.wasm` binary

The execution pipeline: (1) load and compile the `.wasm` module, (2) instantiate per matched node without host imports, (3) resolve the `apply_node` exported symbol, (4) pass the node's `DeviceContext` as serialised JSON. The current implementation validates compilation and symbol resolution; full linear memory I/O for JSON exchange is planned for a future release (see §14).

> **Note**
>
> The `wasm` feature is optional to avoid the `wasmtime` dependency (which adds ~40 MB to the binary) for users who do not need plugin extensibility.

5.5.13 `build_prefix_list`

Creates named prefix-list entries on matched nodes, stored as `Stanza` objects for downstream template rendering.

Table 17. `build_prefix_list` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for target nodes
`prefix_list_name`	String	Yes	Name of the prefix list
`entries`	Vec<PrefixListEntry>	Yes	Ordered prefix-list entries

Each `PrefixListEntry` has: `prefix` (CIDR string), `action` (`permit`/`deny`), optional `le` (less-than-or-equal prefix length), and optional `ge` (greater-than-or-equal prefix length).

5.5.14 `build_community_list`

Creates named community-list entries on matched nodes.

Table 18. `build_community_list` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for target nodes
`community_list_name`	String	Yes	Name of the community list
`entries`	Vec<CommunityListEntry>	Yes	Ordered community-list entries

Each `CommunityListEntry` has: `community` (e.g. `65001:1000`) and `action` (`permit`/`deny`).

5.5.15 `generate_safe_bgp_filters`

Auto-generates RFC 1918 bogon filters as prefix-list and routing-policy stanzas on matched nodes.

Table 19. `generate_safe_bgp_filters` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for target nodes
`prefix_list_name`	String	No	Name for the generated prefix list
`policy_name`	String	No	Name for the generated routing policy
`block_rfc1918`	bool	No	Block RFC 1918 prefixes (default: true)
`permit_default`	bool	No	Add a trailing permit-any (default: false)

5.5.16 `build_zone_policy`

Compiles a zone-based firewall policy for traffic flowing between two security zones. The operator declares the zone pair, an ordered list of match/action rules, and a default action (deny by default). At execution time, the primitive validates zone names against the group registry, constructs a typed `SecurityModel` on each matched node, and appends the zone policy to the node's `DeviceContext`. Vendor templates render the policy into platform-specific syntax (Junos security-zone ACLs, Cisco ZBFW class-maps, VyOS zone-policy rules, etc.).

Table 20. `build_zone_policy` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for target devices
`from_zone`	String	Yes	Source security zone name
`to_zone`	String	Yes	Destination security zone name
`default_action`	String	No	`permit` or `deny` (default: `deny`)
`rules`	Vec<Rule>	Yes	Ordered list of match/action rules

Each rule has a `name`, an `action` (`permit`/`deny`), and an optional `match` block with fields: `source`, `destination`, `service`, `protocol`, `destination_port`, `icmp_type`. Optional `log` and `stateful` flags control logging and connection-tracking behaviour.

5.5.17 `build_nat_policy`

Attaches NAT policy rules to matched devices. Supports four rule types: `snat` (source NAT via interface or pool), `dnat` (destination NAT / port-forwarding), `nat64` (RFC 6146 stateful IPv6-to-IPv4 translation), and `nptv6` (RFC 6296 IPv6 prefix translation). Rules are ordered and may optionally reference zone pairs (`from_zone`, `to_zone`) for zone-scoped NAT.

Table 21. `build_nat_policy` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for target devices
`rules`	Vec<NatRule>	Yes	Ordered list of NAT rules

Each `NatRule` has a `name`, a `type` (`snat`/`dnat`/`nat64`/`nptv6`), an optional `match` block, and a type-specific parameter block. SNAT rules specify `mode` (`interface` or `pool`) and optional `pool_addresses`. DNAT rules specify `external_address`, optional `external_port`, `internal_address`, and optional `internal_port`. NAT64 rules specify a `nat64_prefix`; NPTv6 rules specify `internal_prefix`, `external_prefix`, and an optional `allow_best_effort` flag. Rules may also carry `from_zone`/`to_zone` for zone-scoped NAT and an optional `log` flag.

The `match` block accepts four optional fields:

Table 22. NAT rule match fields (`NatMatchSpec`).

Field	Type	Description
`source`	String	Source CIDR or `$address_object_name`
`destination`	String	Destination CIDR or `$address_object_name`
`service`	String	`$service_object_name` or bare identifier
`protocol`	String	IP protocol (e.g. `tcp`, `udp`)

5.5.18 `build_vxlan`

Assigns VXLAN tunnel bindings to matched nodes. VNIs are allocated sequentially from `vni_base`.

Table 23. `build_vxlan` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for VTEP nodes
`vni_base`	u32	Yes	Starting VNI (e.g. `10000`)
`mcast_group_base`	String	No	Multicast group for BUM replication

5.5.19 `build_evpn`

Assigns EVPN route-distinguisher (RD) and route-target (RT) values to matched nodes.

Table 24. `build_evpn` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for EVPN nodes
`route_distinguisher_base`	String	Yes	RD base (e.g. `auto`)
`route_target_base`	String	Yes	RT base (e.g. `65001:10000`)

5.5.20 `tag_nodes`

Stamps flat key–value metadata onto matched nodes. Tags are merged into each node's `data_json` and are available to subsequent selectors and assertions.

Table 25. `tag_nodes` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for target nodes
`tags`	HashMap<String, JsonValue>	Yes	Key–value pairs to stamp

5.5.21 `map_hardware_inventory`

Assigns chassis model and line-card slot mappings to matched nodes. Used by the TSDM layer (§7.4) via CEL extension functions (e.g., `hw.port_name(node.chassis, slot)`) to resolve abstract interface names into physical slot/port identifiers.

Table 26. `map_hardware_inventory` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for target nodes
`chassis_model`	String	Yes	Chassis identifier (e.g. `dcs-7508`)
`slots`	HashMap<u32, String>	Yes	Slot → line-card model mapping

5.5.22 `assert_state`

A mid-pipeline assertion checkpoint. Syntactically identical to a top-level `AssertionSpec` but placed inline within a layer's primitive list. This allows the operator to validate intermediate state between primitives rather than waiting until the post-execution assertion pass.

Table 27. `assert_state` fields.

Field	Type	Required	Description
`name`	String	Yes	Assertion name (for diagnostics)
`severity`	String	No	`error` (default) or `warning`
`select`	String	Yes	CEL selector for items to check
`check`	AssertionCheck	Yes	Predicate (same types as top-level assertions)
`help`	String	No	Remediation hint shown on failure

5.6 Design-rule assertions

Blueprints may declare assertions: post-execution invariants checked after all primitives complete but before commit. Each assertion has a severity: `error` (pipeline failure) or `warning` (diagnostic only).

Listing 8. Design-rule assertions in a blueprint.

```yaml
assertions:
  - name: "all routers have hostname"
    severity: error
    select: all()
    check: { type: field_exists, field: hostname }
  - name: "IPs within allocation"
    severity: error
```

```yaml
    select: "nodes[src_ip != null]"
    check:
      field_in_cidr:
        field: src_ip
        cidr: "10.0.0.0/8"
  - name: "unique hostnames per site"
    severity: warning
    select: all()
    check:
      unique_per_group:
        field: hostname
        group_by: site
  - name: "private ASN range"
    severity: warning
    select: "nodes[asn != null]"
    check:
      custom_cel:
        expression: "asn >= 64512 && asn <= 65534"
```

Seventeen check types are available:

Table 28. Assertion check types.

Check	Semantics
`field_exists`	Every matched node has the named field in its `data_json`
`min_count`	The matched set contains at least N items
`max_count`	The matched set contains at most N items
`min_edges`	Every matched node has at least N outgoing edges
`unique_per_group`	Within groups defined by `group_by`, the `field` value is unique
`field_in_cidr`	Every matched node's IP field is contained within the specified CIDR
`custom_cel`	Arbitrary CEL expression evaluated per matched entity; fails if false
`use_rule`	Reference a named CEL macro from the `rules:` section
`is_connected`	The matched subgraph (nodes + interconnecting edges) is connected
`is_acyclic`	The matched subgraph contains no cycles
`is_bipartite`	The matched subgraph is bipartite (two-colourable)
`reachability`	Every matched node can reach at least one node in a target selector
`match_schema`	Each matched node's `data_json` validates against a JSON Schema
`zone_membership`	Every matched node belongs to one of the declared security zones
`zone_policy_exists`	A zone policy exists for the specified from/to zone pair
`no_shadowed_rules`	No rule in a zone policy is fully shadowed by a preceding rule
`zone_pairs_covered`	All declared zone pairs have corresponding zone policies

This enables "policy as code" directly in the blueprint. Assertions are evaluated cross-layer (no layer filter), so a single assertion can validate properties across both physical and protocol layers.

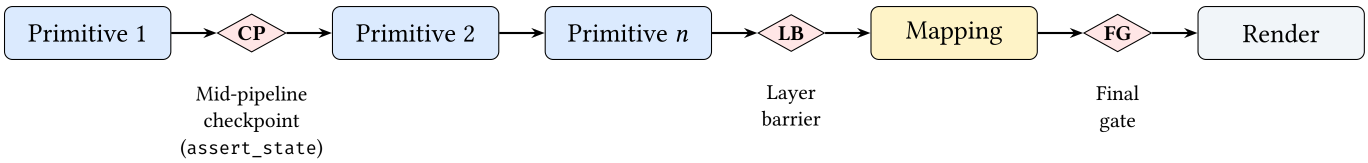

Figure 12. Assertion firing points. **CP**: mid-pipeline checkpoint (inline `assert_state`); **LB**: layer barrier (runs after a layer's primitives complete); **FG**: final gate (runs before rendering). All three evaluate the same assertion types but fire at different pipeline positions for earlier feedback.

5.7 Named rules (CEL macros)

The `rules:` section defines named CEL macros that can be referenced by assertions via the `use_rule` check type. This avoids duplicating complex CEL expressions across multiple assertions.

Listing 9. Named rules and their use in assertions.

```yaml
rules:
  tenant-assigned: "has(node.tenant) && node.tenant != ''"
  vrf-configured:  "has(node.vrf_name) && node.vrf_name != ''"
  leaf-has-vlan:   "has(node.vlan_id) && node.vlan_id > 0 && node.vlan_id < 4095"
```

Rules are simple string-valued CEL expressions stored in a `HashMap<String, String>`. At evaluation time, a `use_rule` check looks up the named rule and evaluates it as though it were an inline `custom_cel` expression. Rules from imported files are merged into the importing blueprint's rule namespace.

5.8 Blast radius policies

The `blast_radius:` section declares change-safety policies that constrain how much the topology may change in a single commit. Each policy specifies a check against the computed `MutationPlan`:

Table 29. Blast radius check types.

Check	Semantics
`max_deletions`	At most N nodes or edges may be deleted
`max_modifications`	At most N nodes or edges may be modified
`max_percentage_changed`	At most P% of the topology may be altered (0.0–100.0)

An optional `select` field restricts the policy scope to a subset of the topology; when omitted, the policy applies to the entire mutation plan.

Listing 10. Blast radius policies.

```yaml
blast_radius:
  - name: tenant-vrf-protection
    check:
      type: max_deletions
      count: 3
  - name: fabric-change-cap
    check:
      type: max_percentage_changed
      value: 25.0
```

5.9 Target-specific transforms

The `transforms:` section declares AST-to-AST rewrite rules that adapt vendor-neutral stanzas into platform-specific syntax (§7.4 details the execution engine). Each `TransformationSpec` has a `name`, an optional `when` predicate (CEL expression evaluated per device), and a sequence of `RewriteRule` entries:

Table 30. Transform rewrite rule fields.

Field	Type	Required	Description
`name`	String	Yes	Transform name
`when`	String	No	CEL predicate (e.g. `device_os == 'nxos'`)
`rules`	Vec<RewriteRule>	Yes	Sequence of match/apply rewrites

Each `RewriteRule` has a `match_expr` (CEL expression selecting stanzas) and an `apply` map (field name → CEL expression computing the new value):

Listing 11. Vendor-specific transform.

```yaml
transforms:
  - name: nxos-interface-rewrite
    when: "device_os == 'nxos'"
    rules:
      - match_expr: "stanza.kind == 'interface'"
        apply:
          name: "'Ethernet1/' + string(stanza.fields.abstract_index + 1)"
```

6 Compilation Pipeline

NetCfg's compilation pipeline has five distinct layers of abstraction, operating as a true network compiler. Figure 13 shows the data flow from abstract intent to concrete syntax.

A key property of this pipeline is **determinism**: configuration is a pure function $f(\text{Intent}, \text{Topology}) = \text{DeviceIR} \to \text{Config}$. If the intent has not changed, regenerating the configuration produces a bit-identical result, making any divergence immediately detectable via diff. Combined with the structural diff engine (§7.3), this enables a "regenerate and compare" workflow for detecting configuration drift without runtime monitoring.

> **Note**
>
> The conference paper describes six implementation stages (parse, shadow, execute, map, render, write); this report groups these into five conceptual layers (Intent, Assertions, Mapping/DeviceIR, TSDM, Syntax). Both describe the same pipeline at different levels of abstraction: the conference paper's "parse" and "shadow" stages are subsumed by Layer 1 (Intent Transformation), and "render" and "write" are subsumed by Layer 5 (Syntax Serialisation).

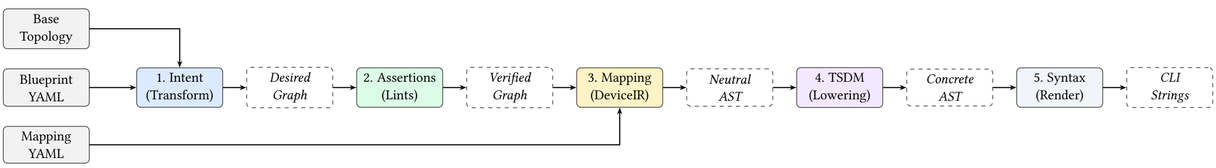

Figure 13. NetCfg compilation pipeline. High-level intent is progressively lowered through three distinct AST representations (Verified Graph, Neutral AST, Concrete AST) before serialisation.

6.1 Layer 1: Intent Transformation

The blueprint is parsed into an AST and applied to a transactional *shadow* copy of the topology (Figure 14). The shadow is created via a serialisation round-trip, providing an isolated working copy: if any primitive or assertion fails, the shadow is discarded and the production graph remains untouched.

Composable primitives execute in declaration order within each layer. Topology primitives (`mesh_nodes`, `build_protocol_layer`) create graph structure; allocation primitives (`provision_ips`, `allocate_resources`) assign derived identifiers; policy primitives (`build_zone_policy`, `build_routing_policy`) compile intent into per-device rule objects. Each primitive follows the *select* → *enrich* → *validate* pattern (§2.6), and each sees the enriched model left by its predecessors.

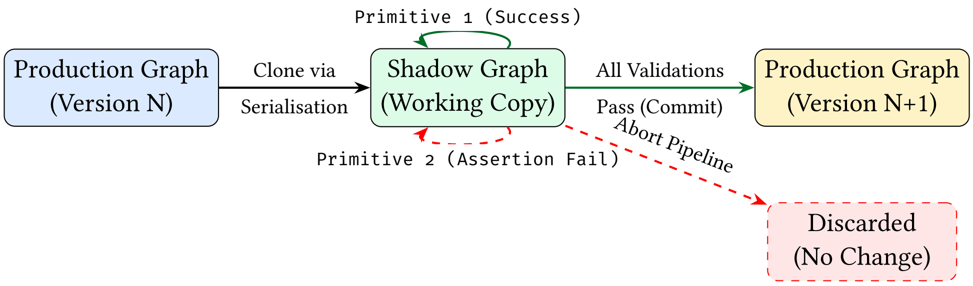

Figure 14. Transactional Safety via the Shadow Topology. Mutations occur on an isolated clone. If any primitive or assertion fails, the shadow is discarded without side-effects on the production graph.

6.2 Layer 2: Topological Assertions

After all primitives complete, the DSL-driven assertion engine (§5.6) evaluates every assertion declared in the blueprint against the enriched graph. Assertions are typed checks—field existence, count bounds, CIDR containment, graph connectivity, zone-pair coverage, and arbitrary CEL predicates—evaluated cross-layer so that a single assertion can span physical and protocol layers simultaneously.

Assertions with severity `error` abort the pipeline; those with severity `warning` emit diagnostics but allow compilation to continue. This layer acts as the quality gate between intent transformation and lowering: only a fully validated model proceeds to DeviceIR generation.

6.3 Layer 3: DeviceIR and the Mapping DSL

Between stage 2 (Assertions) and stage 4 (TSDM) sits the Mapping Evaluation engine. A Mapping DSL lowers the rich topology graph into a vendor-neutral **Abstract Syntax Tree (DeviceIR)**. This representation captures logical intent (e.g. "Neighbor X is remote-as Y") without hardware context.

A mapping rule consists of a `selector` and a `stanza` template. When evaluated against the graph, it emits discrete configuration units (e.g., interfaces, BGP neighbours) that are attached to each node's `DeviceContext.stanzas` list (Figure 15).

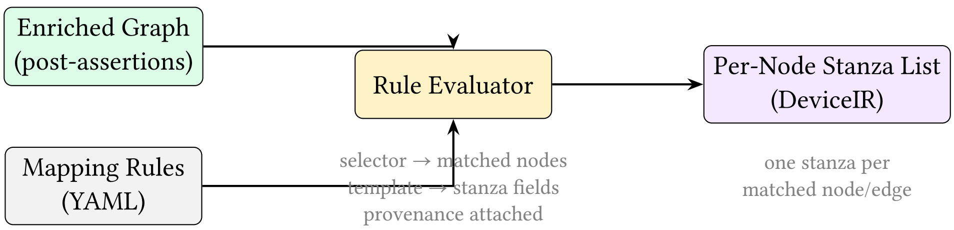

Figure 15. Mapping evaluation. Each rule's selector matches a subset of the enriched graph; the stanza template extracts fields into a structured DeviceIR unit attached to the node's `DeviceContext`.

Listing 12. Mapping DSL extracting an interface stanza.

```
rules:
  - selector: "edges[layer=='physical']"
    stanza:
      kind: "interface"
      fields:
        name: "{{ node.peer_interfaces[peer.hostname] }}"
        description: "Link to {{ peer.hostname }}"
        mtu: 9000
```

6.4 Layer 4: Target-Specific Transform (TSDM)

The **TSDM Layer** performs AST-to-AST transformations (Figure 16). It lowers the Neutral AST into a platform-specific AST, remapping logical interfaces to physical ports (e.g. `Eth1` → `Eth1/1/1`) based on the target's chassis model and hardware layout.

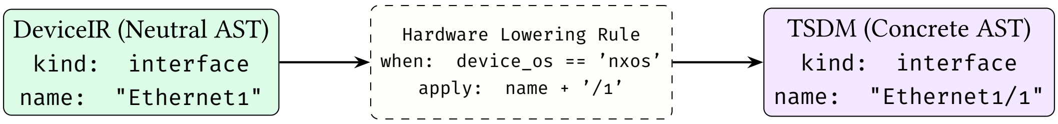

Figure 16. AST-to-AST lowering via TSDM. Vendor-neutral DeviceIR is transformed into a target-specific data model using rules evaluated against the device's metadata.

6.5 Layer 5: Syntax Serialisation

MiniJinja templates act as a *dumb emission layer*, serialising the final Target-Specific AST into vendor-specific CLI syntax. Templates contain no conditional logic beyond iteration over stanza lists and field interpolation—all architectural decisions have already been resolved by the preceding layers. This keeps templates thin and auditable: a template bug can only affect formatting, never topology or policy semantics.

The `TransactionalOutput` writer collects all files in temporary locations (using `NamedTempFile` in the same directory for same-filesystem guarantee), then atomically renames all via POSIX `rename(2)` on commit. If any stage fails, all temp files are dropped without committing—no partial output directory is ever visible.

Three artifacts are emitted per node:

- `{hostname}.conf`: the rendered device configuration.
- `{hostname}.deviceir.json`: the raw DeviceIR stanzas (audit).
- `{hostname}.lineage.json`: per-line mapping from config lines to the `rule_id` that produced them (provenance tracing).

6.5.1 Determinism guarantee

Given a fixed blueprint B, mapping M, and input topology G, the output configuration set $C = compile(G, B, M)$ is deterministic. Serialisation round-trip produces a bit-identical shadow. Primitives execute in declaration order. Selectors evaluate deterministically. Node ID allocation is monotonic. IP allocation iterates edges sorted by (src, dst). DeviceIR stanzas use `BTreeMap` and are post-sorted. Template rendering is pure.

7 Core Mechanisms

7.1 Five-pass IPAM (`provision_ips`)

The IP allocator (`primitives/ipam.rs`) implements a five-pass algorithm:

1. **State discovery**: Scan all nodes for existing IP assignments (`src_ip`, `dst_ip`, `subnet` fields).
2. **Edge selection**: Evaluate the CEL selector; sort matched edges by (src, dst) for determinism.
3. **Pre-reservation**: Insert existing allocations into the allocator, preventing re-allocation and enforcing stickiness.
4. **Fulfilment**: For each edge, allocate a subnet using the configured strategy (`dense`, `sparse`, or `hash`). Extract host addresses from the subnet.
5. **Write-back**: Write `src_ip`, `dst_ip`, `subnet` (and IPv6 equivalents for dual-stack) into each endpoint's `DeviceContext`.

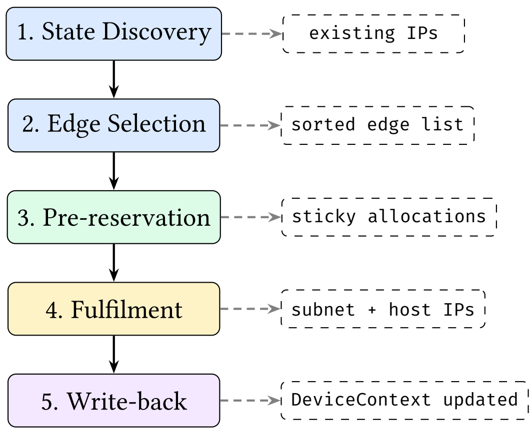

Figure 17. Five-pass IPAM algorithm. Passes 1–3 gather state and reserve existing allocations; pass 4 allocates new subnets; pass 5 writes results to `DeviceContext`. The two-phase structure (read then write) satisfies Rust's ownership rules without `unsafe` code.

Identity-based stickiness. The allocator uses a canonical identity —`sorted(hostname_a, hostname_b)`—as the primary key. If a prior allocation exists for the same identity, the same prefix is returned without pool search. This anchors allocations to stable semantic identity, not integer IDs. If topology IDs change (e.g., after a re-import) but hostnames remain the same, allocations are preserved.

VRF isolation. The allocator maintains per-VRF state. Two primitives using the same CIDR pool in different VRFs can allocate overlapping subnets without conflict. Pool overlap *within* the same VRF is detected and rejected.

Allocation strategies:

- `dense`: Sequential allocation from the pool start. First edge gets the first available subnet.
- `sparse`: Skips every other subnet, leaving growth headroom between allocations.
- `hash`: Subnet position is derived from a hash of the edge identity, producing topology-independent allocations that do not depend on evaluation order.

Hierarchical pools. A `global_pool` primitive registers a large CIDR (e.g., `10.0.0.0/8`). Subsequent `provision_ips` primitives reference the pool by name and specify a `pool_size` (e.g., 24), carving /24 blocks from the parent pool on demand.

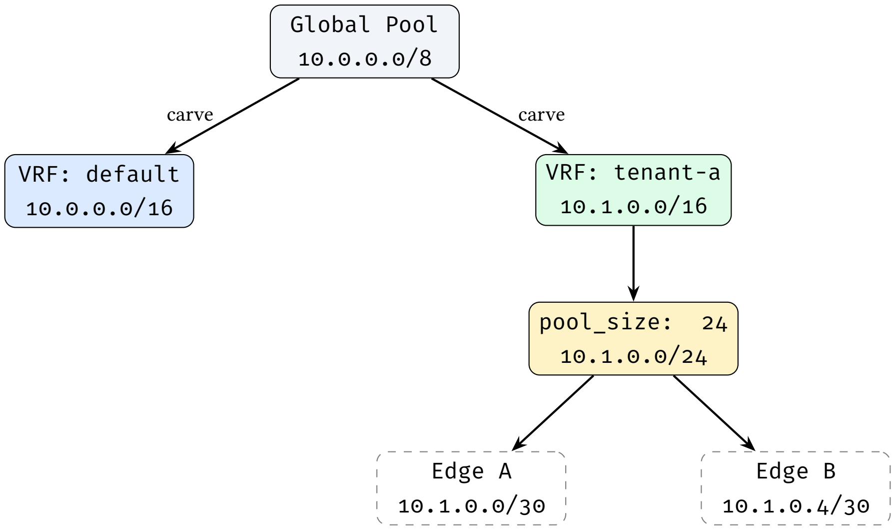

Figure 18. Hierarchical IP pool carving with VRF isolation.

Dual-stack. When `ipv6_pool` is specified, both IPv4 and IPv6 addresses are allocated in a single primitive invocation. IPv6 fields (`src_ipv6`, `dst_ipv6`, `subnet_v6`) are written alongside IPv4 fields.

7.2 Protocol overlay construction (`build_protocol_layer`)

`build_protocol_layer` clones physical nodes into a named protocol layer. For each selected source-layer node:

1. Allocate a new monotonic node ID.
2. Add the node to the target layer.
3. Deep-merge the config overlay into the cloned node's metadata.
4. Record `_source_id` as a parent mapping. For MLAG/vPC setups, `_source_id` can be an array mapping one logical protocol node to multiple physical host switches.
5. Optionally (`clone_underlying: true`) clone edges between source-layer nodes into the overlay.

Cross-layer traceability: the `_source_id` field links each protocol node to its physical origin. Since `provision_ips` already enriched the physical node's `DeviceContext` with IP addresses, the cloned protocol node inherits them via deep merge—this is how IP addresses flow from IPAM allocation to BGP configuration without explicit wiring.

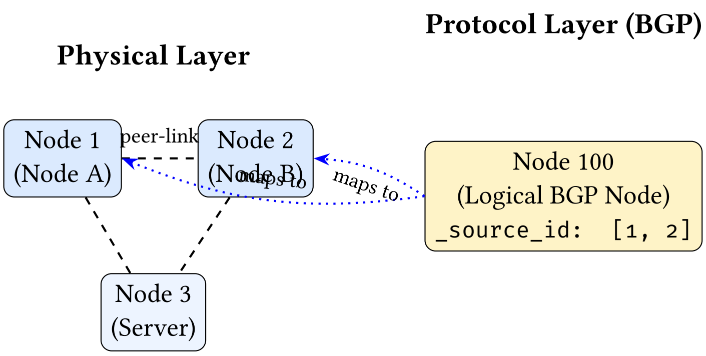

Figure 19. MLAG/vPC topology mapping. Two physical nodes collapse into a single logical protocol node via array `_source_id`.

Deep merge semantics: When both the base node and the config overlay contain nested JSON objects under the same key, the merge recurses key-by-key (overlay wins on conflict). Arrays are replaced entirely rather than concatenated, ensuring idempotency. This preserves sibling keys that the overlay does not mention. For example, a base node with `{bgp: {as: 65000, holdtime: 180}}` and an overlay with `{bgp: {timers: {keepalive: 60}}}` produces `{bgp: {as: 65000, holdtime: 180, timers: {keepalive: 60}}}`.

Idempotency: The primitive scans the target layer for existing nodes with `_source_id` matching source nodes. Already-cloned nodes are skipped, so re-running the primitive after a partial failure produces the correct result.

7.3 Configuration diffing (DiffEngine)

The diff engine (`diff/engine.rs`) computes a `MutationPlan` between two topologies by exporting each layer to JSON and comparing:

1. **Node diff**: Per layer, compare node sets by a composite semantic key `(layer, hostname, protocol_type)`. Diffing by integer ID is avoided, as node insertions would shift downstream IDs and create massive false diffs. Detect additions, deletions, and modifications (field-by-field property comparison via `extract_properties`, which unpacks the `data_json` blob).
2. **Edge diff**: Compare by (src, dst) pair with structural property comparison.

The `MutationPlan` contains six change types: `AddNode`, `RemoveNode`, `UpdateNode`, `AddEdge`, `RemoveEdge`, `UpdateEdge`. Changes are sorted deterministically by (change_type, layer, id/src/dst) for reproducible output.

The `plan` command displays the plan as a human-readable diff; the `ci-run` command uses it to compute per-device configuration diffs.

7.4 AST Transformations (TSDM Layer)

The **Target-Specific Device Model (TSDM)** layer implements the compiler's lowering phase via DSL-driven **AST-to-AST transformations**, whose blueprint syntax is described in §5.9. This section details the execution engine that bridges logical design and physical hardware constraints.

7.4.1 The Transformation DSL

Lowering rules are defined in the blueprint using an expressive rewrite-rule grammar. Each rule consists of a `match_expr` (CEL predicate) and an `apply` map (field mutations).

Example: NX-OS interface lowering

```yaml
transforms:
  - name: "nxos_lowering"
    when: "device_os == 'nxos'"
    rules:
      - match_expr: "kind == 'interface' && name.startsWith('Ethernet')"
        apply:
          name: "name + '/1'"  # Ethernet1 -> Ethernet1/1
          mtu: "9216"          # Force jumbo frames
```

7.4.2 Execution Logic

The transformation engine (`dsl/transform.rs`) performs a deep-walk of the DeviceIR stanzas. For each stanza, it:

1. Binds the stanza's fields as CEL variables.

2. Evaluates the `match_expr` predicate.
3. If matched, executes the `apply` expressions to mutate the stanza's state.

This structured approach allows complex hardware logic—such as mapping logical ports to specific line-card slots in a modular chassis—to be expressed as programmable transformations rather than hardcoded logic or brittle templates.

The key architectural insight is that `DeviceContext` accumulates state across the entire primitive pipeline:

1. `mesh_nodes`: writes `mesh_type`, `peer_count`, and optionally `peer_interfaces` into metadata.
2. `provision_ips`: writes `src_ip`, `dst_ip`, `subnet`, `vrf`.
3. `build_protocol_layer`: deep-merges all of the above into the cloned protocol node, plus the config overlay.
4. `allocate_resources`: writes the allocated resource value (e.g., `asn`) into metadata.

By the time Stage 4 (rendering) occurs, each node's `DeviceContext` carries hostname, IPs, protocol metadata, interface assignments, and any custom fields—the complete information needed to produce a device configuration file.

8 CLI Reference

NetCfg provides twelve CLI commands spanning the development-to-deployment lifecycle. All commands use `clap` for argument parsing. Table 31 shows which pipeline layers each command invokes.

Table 31. CLI commands and the pipeline layers they invoke. ✓= executes; ◦ = optional (via flag).

Command	L1 Intent	L2 Assert	L3 Mapping	L4 TSDM	L5 Render	Primary output
`generate`	✓	✓	✓	✓	✓	Device config files
`validate`	✓	✓			◦	Pass/fail exit code
`plan`	✓	✓			◦	Mutation diff
`inspect`	◦					AST / DOT / trace
`ci-run`	✓	✓	✓	✓	✓	JSON report
`lint`	✓	✓				Assertion results
`impact`	✓	✓				Risk assessment
`report`	✓	✓				Markdown report
`fmt`						Formatted YAML
`import`						.nte archive
`export-clab`	✓	✓				clab.yaml
`sync`						.nte archive / stdout

8.1 `netcfg generate`

Runs the full five-layer compilation pipeline (§6), writing artefacts to disk.

```
netcfg generate <topology> <blueprint> \
    --templates-dir <DIR> \
    [--output-dir <DIR>] [--emit-ir] [--verbose]
```

Argument	**Default**	**Description**
`topology`	—	Path to base topology archive (.nte)
`blueprint`	—	Path to blueprint YAML
`–templates-dir`	—	Directory containing `.j2` templates
`–output-dir`	`./output`	Output directory for generated files
`–emit-ir`	false	Also write `{hostname}.ir.json`
`–verbose`	false	Print per-node rendering progress

Output: one `.cfg` file per device (plus `.ir.json` if `–emit-ir`). Absolute paths are printed to stdout, one per line, suitable for piping.

8.2 `netcfg validate`

Executes the pipeline in dry-run mode: all primitives run with `dry_run=true`, skipping DataFrame writes but verifying what *would* change. Optionally pre-flights templates by rendering to memory.

```
netcfg validate <topology> <blueprint> \
    [--templates-dir <DIR>] [--verbose]
```

Exit codes: 0 on success, non-zero on validation failure (parse errors, assertion failures, template syntax errors, IP pool conflicts).

8.3 `netcfg plan`

Computes the topology `MutationPlan` (current vs. desired) and displays additions, removals, and property changes. With `–diff`, it renders before/after configs and shows a unified text diff per device using the `similar` crate.

```
netcfg plan <topology> <blueprint> \
    [--diff] [--templates-dir <DIR>] [--verbose]
```

Output without `–diff`: colourised plan (green for additions, red for removals, yellow for modifications). Summary line: `+N additions, -N removals across M layers`.

Output with `–diff`: unified diff per device, preceded by `diff –cfg {hostname}`.

8.4 `netcfg inspect`

Provides three views into intermediate state:

```
netcfg inspect <path> --ast                # Print blueprint AST
netcfg inspect <path> --topology           # Export as Graphviz DOT
    [--blueprint <FILE>]
netcfg inspect <path> --trace <NODE_ID>    # Trace node state changes
    --blueprint <FILE>
```

- `–ast`: Parses the file as a blueprint and prints the AST tree.

- `–topology`: Exports the graph as Graphviz DOT. With `–blueprint`, applies the blueprint first. Output includes per-layer subgraph clustering, hostname labels, and subnet annotations.
- `–trace <node_id>`: Replays primitive execution and prints before/after `DeviceContext` snapshots for the specified node at each primitive step. When blueprints fail with selector or assertion errors, finding the root cause in a 1,000-node graph is challenging; `–trace` is the primary debugging workflow for blueprint authors.

8.5 `netcfg ci-run`

The CI/CD integration point. Runs validate, plan, and config diff in a single invocation.

```
netcfg ci-run <topology> <blueprint> \
    --templates-dir <DIR> [--json]
```

With `–json`, emits a structured payload:


```
{
  "status": "success",
  "topology_additions": 42,
  "topology_removals": 0,
  "configuration_diffs": {
    "core-0": "-  ip address 10.0.0.4/30\n+  ip address 10.0.0.8/30"
  },
  "warnings": []
}
```


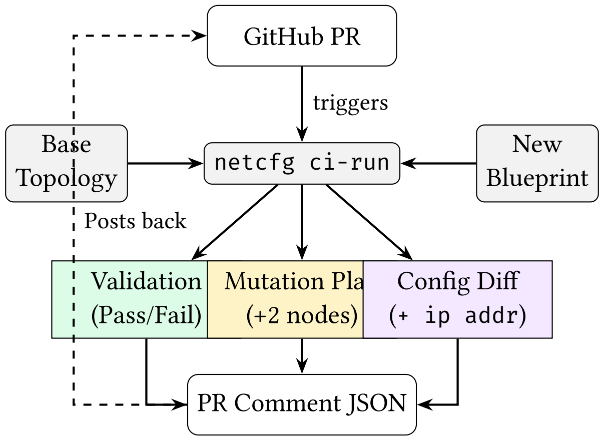

Figure 20. The NetDevOps CI/CD pipeline enabled by `ci-run`.

This integrates with GitHub Actions, GitLab CI, or any CI system that can parse JSON.

8.6 `netcfg import`

Imports a YAML topology definition into an NTE archive.

```
netcfg import <yaml_path> <output.nte>
```

The importer reads a topogen-format YAML file (defining nodes with types, layers, and properties) and serialises it to an `.nte` archive that other commands consume.

8.7 `netcfg lint`

Runs design-rule assertions without the full validation pipeline. Applies the blueprint in dry-run mode, then evaluates assertions and reports results.

```
netcfg lint <topology> <blueprint> [--verbose]
```

Returns assertion results with pass/fail status, severity, and diagnostic messages. Exit code 1 only when `error`-severity assertions fail; warnings are reported but do not cause failure.

8.8 `netcfg fmt`

Standardises blueprint YAML formatting. Parses the blueprint via `Executor::parse()`, re-serialises with `serde_yml`, and compares byte-for-byte. Idempotent.

```
netcfg fmt <blueprint> [--check] [--verbose]
```

With `–check`, exits with code 1 if formatting changes are needed (useful as a pre-commit hook or CI gate). Without `–check`, overwrites the file in place.

8.9 `netcfg impact`

Calculates the operational blast radius of a blueprint change against a base topology. The topology argument accepts either a local `.nte` archive path or an NSoT URI (e.g. `netbox://host?site=ldn1`); see `netcfg sync` (§8.12) for URI syntax.

```
netcfg impact <topology> <blueprint> [--detail] [--verbose]
```

The `–detail` flag expands the low-risk section to list individual nodes (by default only a count is shown).

For each mutation in the computed plan, assigns a risk level:

Risk	**Colour**	**Trigger**
High	Red	Node removal; critical property change (`asn`, `router_id`, `src_ip`, `subnet`); physical link add/remove
Medium	Yellow	Node addition; logical link add/remove
Low	Green	Metadata-only update

Critical properties are: `asn`, `router_id`, `src_ip`, `dst_ip`, `subnet`, `loopback`, `mgmt_ip`. Output includes per-node risk attribution with reasons.

8.10 `netcfg report`

Generates an automated Markdown architecture report from a compiled topology.

```
netcfg report <topology> <blueprint> -o <output.md> [--verbose]
```

The report contains three sections:

1. **Node inventory**: hostname, type, OS, ASN, loopback, management IP.
2. **Physical links & IP addressing**: source/destination hosts, interfaces, IPs, subnets.
3. **Topology visualisation**: A `netviz-json` code block with a custom JSON schema (`NetVizNode`, `NetVizEdge`, `NetVizTopology`) suitable for rendering by external visualisation tools.

8.11 `netcfg export-clab`

Generates a Containerlab `clab.yaml` simulation file from a compiled topology, enabling rapid lab deployment for testing (e.g. the three-site mesh from §9). The topology argument accepts a local `.nte` archive or an NSoT URI (§8.12).

```
netcfg export-clab <topology> <blueprint> \
    -o <clab.yaml> [--configs-dir <DIR>] [--verbose]
```

Heuristically maps `device_os` to Containerlab container kinds: `cisco_iosxr` → `vr-xrv9k`, `nokia_srl` → `srl`, `arista_eos` → `ceos`, `default` → `linux`. When `–configs_dir` is provided, bind-mounts generated configuration files into OS-specific remote paths (e.g. `/etc/opt/srlinux/config.json` for SRL, `/mnt/flash/startup-config` for cEOS).

8.12 `netcfg sync`

Synchronises topology data with external Network Source of Truth (NSoT) systems. Two subcommands are available:

```
netcfg sync pull <uri> [-o <output.nte>] [--token <TOKEN>] [--insecure]
netcfg sync push <uri> <source> [--token <TOKEN>]
```

`sync pull` fetches a live topology from an NSoT and converts it into an NTE topology archive. The URI scheme identifies the provider: `netbox://host[:port]?site=X&region=Y` targets a NetBox [6] instance via its GraphQL endpoint; query parameters are passed as filters. Authentication is resolved in order: the `–token` flag, then the `NETBOX_TOKEN` environment variable. When `-o` is supplied the topology is persisted to disk; otherwise a summary (node and edge counts) is printed. The `–insecure` flag disables TLS certificate verification for self-signed development instances.

`sync push` is a planned stub that will write intent back to the NSoT (see §12).

> **Note: NSoT URI in other commands**
>
> The topology argument of `impact` (§8.9) and `export-clab` (§8.11) also accepts a `netbox://` URI. Internally these commands dispatch through the same `load_topology_source()` helper, so a live NSoT topology can be used without a local archive.

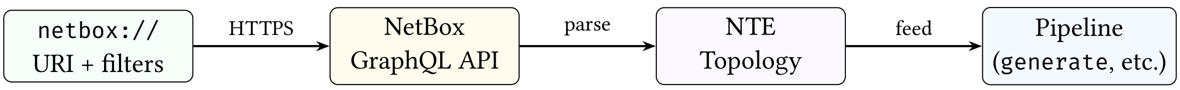

Figure 21. NSoT synchronisation data flow.

9 End-to-End Worked Example

We trace the `three-site-mesh.yaml` blueprint through every pipeline stage. The six stages below correspond to the five pipeline layers in §6: Stages 1–3 are subsumed by Layer 1 (Intent Transformation), Stage 4 maps to Layer 3 (DeviceIR), Stage 5 to Layer 5 (Serialisation), and Stage 6 is the atomic write that concludes Layer 5.

The base topology has 12 routers across 3 sites (4 per site: site-a-r1 through site-c-r4).

9.1 Input: Blueprint

Listing 13. three-site-mesh.yaml (abridged).

```yaml
version: 1
```

```yaml
layers:
  # Stage 1: Full mesh within each site
  - name: input
    primitives:
      - type: mesh_nodes
        selector: "nodes[site='a']"
        mesh_type: full
      - type: mesh_nodes
        selector: "nodes[site='b']"
        mesh_type: full
      - type: mesh_nodes
        selector: "nodes[site='c']"
        mesh_type: full

  # Stage 2: IP allocation per site
  - name: input
    primitives:
      - type: provision_ips
        selector: "edges[src>=0]"
        pool: "10.1.0.0/24"
        subnet_size: 30
        strategy: dense

  # Stage 3: BGP overlay
  - name: input
    primitives:
      - type: build_protocol_layer
        selector: "nodes[true]"
        layer: bgp
        config:
          protocol_type: bgp
          asn_base: "65000"
```

9.2 Stage 1: Parse and validate

The parser validates the YAML, confirms `version: 1`, and compiles each selector string (`nodes[site='a']`, `edges[src>=0]`, etc.) to a CEL program. The normaliser converts `site='a'` to `site=="a"`.

9.3 Stage 2: Shadow topology

The executor clones the base topology via a serialisation round-trip, creating an isolated shadow copy. All subsequent mutations operate on the shadow; the base topology is never modified.

9.4 Stage 3: Primitive execution

mesh_nodes (three invocations): Creates $3 \times \binom{4}{2} = 18$ edges (6 per site). The selector `nodes[site='a']` matches the 4 nodes with `{site: "a"}` in their `data_json`. Each node gets `mesh_type: "full"` and `peer_count: 3` written to its metadata.

provision_ips: The selector `edges[src>=0]` matches all 18 edges. Edges are sorted by (src, dst). The five-pass algorithm allocates /30 subnets from `10.1.0.0/24`:

- Edge (1,2): subnet `10.1.0.0/30`, IPs `10.1.0.1` and `10.1.0.2`
- Edge (1,3): subnet `10.1.0.4/30`, IPs `10.1.0.5` and `10.1.0.6`
- …continuing through all 18 edges.

After this stage, node `site-a-r1` has: `src_ip: "10.1.0.1"`, `subnet: "10.1.0.0/30"`, `vrf: "default"`.

build_protocol_layer: Clones all 12 physical nodes into the `bgp` layer (new IDs 13–24). Each clone receives:

- `protocol_type: "bgp"` and `asn_base: "65000"` (from the config overlay)
- `_source_id: 1` (parent mapping)
- All properties from the physical node, including the IPs written by `provision_ips` (via deep merge)

9.5 Stage 4: Mapping evaluation

The companion mapping (`three-site-mesh-mapping.yaml`) extracts per-device stanzas from the enriched graph:

```
rules:
  - selector: "edges[layer=='physical']"
    stanza:
      kind: "interface"
      fields:
        name: "{{ node.peer_interfaces[peer.hostname] }}"
        description: "Link to {{ peer.hostname }}"
        ip_address: "{{ node.src_ip }}"
        mtu: 9000
  - selector: "nodes[layer=='bgp']"
    stanza:
      kind: "bgp"
      fields:
        local_asn: "{{ node.asn }}"
        router_id: "{{ node.loopback }}"
        neighbors: "{{ node.bgp_peers }}"
```

The evaluator walks each rule's selector, matches the relevant nodes/edges, and emits one stanza per match. Each stanza carries `provenance: {rule_id: "rule_0"}`, linking rendered output back to its source rule.

9.6 Stage 5: Render

Stanzas are grouped by hostname. The Jinja2 template iterates over each node's stanza list and emits vendor-specific CLI syntax. For `site-a-r1`, the rendered output includes:

```
! site-a-r1.conf (abridged)
interface Ethernet0
  description Link to site-a-r2
  ip address 10.1.0.1/30
  mtu 9000
!
interface Ethernet1
  description Link to site-a-r3
  ip address 10.1.0.5/30
  mtu 9000
!
router bgp 65001
  router-id 10.255.0.1
  neighbor 10.1.0.2 remote-as 65001
  neighbor 10.1.0.6 remote-as 65001
```

Each of the 12 routers receives a distinct configuration derived from its position in the topology graph—no two devices share the same output.

9.7 Stage 6: Atomic write

The `TransactionalOutput` writer collects all files in temporary locations, then atomically renames them via POSIX `rename(2)`. Per device, three files are written:

- `site-a-r1.conf`: the rendered configuration
- `site-a-r1.deviceir.json`: the raw stanza list (audit trail)
- `site-a-r1.lineage.json`: per-line provenance mapping

If any stage fails, all temporary files are dropped—no partial output directory is ever visible.

10 Error Model

NetCfg uses typed error enums with `thiserror` derivation and `miette` diagnostic annotations. Table 32 summarises the error types.

Table 32. Error enums and their variants.

Enum	Variants	Context
`ParseError`	`YamlSyntax`, `Validation`, `Deserialize`, `LayerOrdering`	Blueprint parsing
`SelectorError`	`InvalidSyntax`, `Compile`, `Evaluate`, `NonBooleanResult`, `Polars`, `Json`, `TopologyAccess`	Selector compilation and evaluation
`PrimitiveError`	`Validation`, `Execute`, `Selector`, `Topology`, `Graph`, `Polars`, `Json`	Primitive execution
`ExecuteError`	`Parse`, `Primitive`, `Shadow`, `Plan`, `Assertion`	Top-level executor
`ShadowError`	`Clone`, `Commit`, `Validation`	Shadow topology lifecycle
`AssertionError`	`Failed`, `Selector`	Assertion evaluation
`DiffError`	`TopologyAccess`, `ParseError`	Configuration diffing
`RenderError`	`Io`, `Jinja`, `Json`, `MissingTemplate`	Template rendering

`ExecuteError` variants carry `miette::Diagnostic` annotations with diagnostic codes (e.g., `netcfg::parse`, `netcfg::primitive::execution_failed`, `netcfg::assertion`) enabling structured error handling in CI pipelines.

`ParseError::YamlSyntax` includes a source span (`miette::SourceSpan`) pointing at the offending line, enabling IDE-quality error rendering.

11 Editor and Service Integration

11.1 Language Server (LSP)

The `netcfg-lsp` crate provides a Language Server Protocol implementation built on `tower-lsp` and Tokio. It communicates via stdin/stdout and offers three capabilities:

1. **Real-time diagnostics.** On file open, change, and save, the server calls `parse_blueprint()` and maps errors to LSP diagnostics with line-column precision. For `serde_yml` errors (which embed

location as free text), a regex extracts "at line N column M". Diagnostics are published with source label `ank_netcfg` and severity `ERROR`.

2. **Primitive auto-completion.** When the cursor follows a `type:` token (detected by a suffix heuristic), the server offers completion items for all twenty-two primitives, each with a one-line docstring. Trigger characters are space and colon.
3. **Document synchronisation.** Open documents are tracked in a `DashMap<String, String>` (lock-free concurrent map). Full-text sync (`TextDocumentSyncKind::FULL`) is used for correctness. On close, the document is removed and diagnostics are cleared.

11.2 REST API Microservice

The `netcfg-api` crate exposes the compiler as a stateless HTTP service via `axum`, enabling CI/CD pipelines and enterprise orchestrators to compile blueprints without a local `netcfg` binary.

Endpoint: `POST /compile` (port 3000).

Table 33. Compile endpoint request and response schema.

Direction	Field	Type	Description
Request	`topology_zip_base64`	String	Base64-encoded `.nte` archive
Request	`blueprint_yaml`	String	Raw YAML blueprint
Response	`status`	String	`success` or `error`
Response	`configs`	Map<String, String>	Hostname → rendered config
Response	`device_ir`	Map<String, JSON>	Hostname → DeviceContext IR
Response	`warnings`	Vec<String>	Compilation warnings

The service executes in a strict sandbox mode: `inject_secrets` and `wasm_plugin` primitives are disabled by default to prevent remote exfiltration of environment variables. The compilation pipeline decodes the base64 payload into a temporary file, loads the topology, parses the blueprint, executes via `Executor::apply()`, and extracts per-device IR. All processing is in-memory with no persistent state; the service scales horizontally behind a load balancer.

12 Limitations

12.1 `edge_properties` not applied (`mesh_nodes`)

Description. The `MeshNodesSpec.edge_properties` field is accepted by the blueprint parser but intentionally not applied. Applying it requires an edge DataFrame API on `nte_topology::Topology` equivalent to `set_dataframe`/`update_dataframe` but keyed on $(source\_id, target\_id)$ pairs.

Workaround. Edge properties can be encoded as node metadata on both endpoints (e.g., `link_bandwidth` as a per-node field keyed by peer ID).

Planned resolution. Awaiting `nte-topology` ≥0.2 edge DataFrame support. A tracking test (`edge_properties_deferred_until_nte_edge_dataframe`) is present in the test suite.

12.2 Mapping AST inspection not implemented

Description. The `inspect -ast` command can parse and display blueprint ASTs but does not yet support mapping documents. Attempting to inspect a mapping file prints "Mapping AST visualization coming soon."

Workaround. Inspect the mapping YAML directly. The format is simple (a `rules` list with `selector` and `stanza` fields).

Planned resolution. When the mapping DSL grows beyond simple selector/stanza pairs, AST visualisation will be added.

12.3 Shadow validation is a placeholder

Description. The `ShadowTopology::validate()` method currently only checks that the shadow did not become empty. It does not perform structural integrity checks (dangling edges, orphaned protocol nodes, etc.).

Workaround. Design-rule assertions can catch many structural problems. Use `min_edges` and `field_exists` assertions as guardrails.

Planned resolution. Integrate with the NTE policy validation framework when available.

12.4 No intra-primitive rollback

Description. If a primitive fails partway through execution (e.g., `provision_ips` exhausts its pool after allocating half the edges), the shadow topology retains the partial mutations. The pipeline aborts (returning an error), and the shadow is discarded, but there is no mechanism to roll back individual primitives within a shadow.

Workaround. This is usually not a problem: the entire shadow is discarded on error, so no partial state reaches production. The limitation matters only for debugging: the error message may not reflect the full extent of partial mutations.

Planned resolution. No current plan. Intra-primitive rollback would require snapshotting the shadow before each primitive, which doubles memory usage.

12.5 `generate` CLI bypasses mapping document

Description. The `generate` CLI command renders configs directly from `DeviceContext` via templates, bypassing the mapping document. The full pipeline (with mapping evaluation and DeviceIR) is implemented in `config_gen::generate_configs()` but the CLI does not expose it yet.

Workaround. Use the `generate_configs()` function directly from Rust code for the full pipeline with mapping support.

Planned resolution. Add `–mapping` flag to the `generate` CLI command.

12.6 No incremental compilation

Description. Every invocation recompiles the entire topology from scratch. The diff engine can identify which devices changed, but the pipeline cannot skip unchanged devices during compilation.

Workaround. Use `ci-run –json` to compute the diff, then selectively push only the changed configuration files.

Planned resolution. Under investigation. Incremental compilation requires caching intermediate state (the desired topology) between runs and detecting which blueprint changes invalidate which nodes.

12.7 No runtime verification or drift detection

Description. NetCfg is a generation-time tool. It produces configuration artifacts but does not push them to devices, monitor compliance, or detect post-deployment configuration drift. Closed-loop verification requires integration with external tools.

Workaround. Periodically regenerate configurations from the canonical blueprint and diff against deployed state using `ci-run --json`. Use Batfish [10] or gNMI-based telemetry for runtime compliance checking.

12.8 Shadow topology cloning cost

Description. The transactional shadow is created via a serialisation round-trip (`serde_json`), which is $O(n)$ in topology size. For 10,000-node topologies this adds ~200 ms of overhead.

Workaround. Not typically a bottleneck, but for interactive workflows the cost is noticeable. Future work may explore copy-on-write graph snapshots in the NTE engine.

12.9 WASM plugin is experimental

Description. The `wasm_plugin` primitive validates module compilation and symbol resolution but does not implement full linear memory I/O for JSON data exchange between the Rust host and WASM guest. The current implementation is proof-of-concept only.

Workaround. Use `custom_primitive` with nested built-in primitives for extensibility. For complex transformations, implement them as Rust primitives.

Planned resolution. Full C-ABI memory allocation between host and guest is required for production use. See §14.

12.10 Mapping document limitations

Description. Two limitations affect the mapping workflow: (1) the `inspect --ast` command does not yet support mapping documents (§12.2); and (2) there is no auto-generation of mapping rules from OpenConfig YANG schemas or device inventory data—operators must author the mapping document by hand, which requires knowledge of `DeviceContext` field names and stanza structure.

Workaround. The mapping format is intentionally simple (a `rules` list with `selector` and `stanza` fields). Example mappings ship with every case study.

12.11 NSoT push not implemented (`netcfg sync push`)

Description. The `netcfg sync push` subcommand is defined in the CLI but returns an error when invoked. Pushing intent back to an NSoT system requires a write-capable provider interface (e.g. NetBox REST API mutations) that has not yet been implemented.

Workaround. Export the compiled topology to an NTE archive (`sync pull -o`) and ingest it into the NSoT manually.

Planned resolution. A `NetBoxWriter` provider will be added alongside the existing `NetBoxProvider` read path.

13 Related Work

Network configuration synthesis and verification have received significant attention. We position NetCfg against three strands of related work.

Configuration synthesis and formal methods. Early declarative networking languages like Frenetic [15] and NetKAT [16] pioneered the separation of intent from mechanism via formal semantics in SDN, but did not address legacy CLI-driven environments. Propane [17] compiles high-level routing policies written in a purpose-built language into per-device BGP configurations, proving correctness by construction. SyNET [18] uses constraint solving to synthesise router configurations from a formal network specification. Both systems target routing exclusively; NetCfg covers a broader surface (IP allocation, zone policies, NAT, overlays) at the cost of weaker formal guarantees—design-rule assertions rather than proofs. On the data plane, P4 [19] demonstrated that packet-processing hardware could be programmed via a compiler funnel that lowers abstract intent into target-specific architectures. NetCfg applies this exact compiler philosophy to the management plane.

Configuration analysis and verification. Header Space Analysis [20] introduced the foundation for statically checking network properties. Building on this, Batfish [21] models device configurations as a data plane and evaluates reachability queries offline; Minesweeper [22] extends this with symbolic model checking against network-wide invariants. Config2Spec [23] mines specifications from existing configurations to bootstrap verification. Forward Enterprise [24] provides commercial intent verification with a digital-twin approach. These tools analyse *existing* configurations, whereas NetCfg operates upstream: it *generates* the configurations that such tools would later verify, and its assertion framework provides lightweight invariant checking at compile time.

Operational tooling and orchestration. Ansible [8], Salt [25], and Nornir [9] are widely deployed for configuration deployment and orchestration. Cisco NSO [26] provides model-driven (YANG-based) service orchestration. netlab [27] generates lab topologies for testing. These tools focus on *deploying* or *simulating* configurations rather than synthesising them from intent. Frenetic [15] introduced a functional network programming language for SDN controllers, inspiring the declarative philosophy that NetCfg applies to traditional device configuration.

NetCfg occupies a middle ground: it is more expressive than pure routing synthesisers, less formally rigorous than model checkers, and operates at a higher abstraction level than deployment tools. Its graph-based compilation pipeline (§6) and pluggable primitive architecture (§5.5) are, to our knowledge, unique in this space.

14 Future Work

- **GPU-accelerated primitive execution**: Offloading selector evaluation and IPAM allocation to the GPU via NTE's WebGPU compute backend, targeting 100K+ node topologies.
- **WASM linear memory I/O**: The current `wasm_plugin` primitive validates module compilation and symbol resolution but does not implement full linear memory I/O for JSON data exchange. Full C-ABI memory allocation between the Rust host and WASM guest is required for production use.
- **API template rendering**: The `POST /compile` endpoint currently returns placeholder configuration text; integrating the MiniJinja rendering engine into the API service is planned.
- **Mapping CLI integration**: Exposing the full pipeline (including mapping evaluation) through the `generate` CLI command.
- **Incremental compilation**: Caching intermediate state to avoid recompiling unchanged portions of the topology.
- **LSP assertion integration**: Extending the language server to evaluate design-rule assertions in real time, providing inline warnings for selector violations and IP pool capacity.

References

[1] S. Shenker, "The future of networking, and the past of protocols," in *Open Networking Summit*, vol. 20, Palo Alto, CA: Open Networking Foundation, 2011, pp. 1–30.

[2] P. Gill, N. Jain, and N. Nagappan, "Understanding network failures in data centers: Measurement, analysis, and implications," in *ACM SIGCOMM computer communication review*, vol. 41, New York, NY, USA: ACM, 2011, pp. 350–361.

[3] J. Zhu et al., "MALT: Multi-abstraction layer topology," in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, Santa Clara, CA: USENIX Association, 2020, pp. 313–328.

[4] OpenConfig Working Group, *OpenConfig*, `https://openconfig.net`, 2023.

[5] M. Bjorklund, "The YANG 1.1 data modeling language," IETF, RFC 7950, 2016.

[6] NetBox Community, *NetBox: The source of truth for network automation*, `https://netbox.dev/`, 2023.

[7] Network to Code, *Nautobot: Network source of truth and automation platform*, `https://nautobot.com/`, 2023.

[8] Red Hat, *Ansible automation platform*, `https://www.ansible.com/`, 2023.

[9] Nornir Automation, *Nornir: Pluggable multi-threaded framework with inventory management to help operate collections of devices*, `https://nornir.tech/`, 2023.

[10] Intentionet, *Batfish: Network configuration analysis tool*, `https://www.batfish.org/`, 2023.

[11] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *International Symposium on Code Generation and Optimization, 2004. CGO 2004.*, IEEE, San Jose, CA: IEEE, 2004, pp. 75–86.

[12] Google, *Common Expression Language (CEL)*, `https://github.com/google/cel-spec`, 2023.

[13] S. Knight, *AutoNetKit NTE: The network topology engine*, 2026.

[14] Bytecode Alliance, *Wasmtime: A fast and secure runtime for WebAssembly*, `https://wasmtime.dev/`, 2024.

[15] N. Foster et al., "Frenetic: A network programming language," in *Proceedings of the 16th ACM SIGPLAN International Conference on Functional Programming*, 2011, pp. 279–291.

[16] C. J. Anderson et al., "NetKAT: Semantic foundations for networks," in *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 2014, pp. 113–126.

[17] R. Beckett, R. Mahajan, T. Millstein, J. Padhye, and D. Walker, "Propane: A general approach to network configuration synthesis," in *Proceedings of the 2016 ACM SIGCOMM Conference*, 2016, pp. 440–453.

[18] A. El-Hassany et al., "Network configuration synthesis with logic-based routing models," in *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2017.

[19] P. Bosshart et al., "P4: Programming protocol-independent packet processors," in *ACM SIGCOMM Computer Communication Review*, vol. 44, ACM New York, NY, USA, 2014, pp. 87–95.

[20] P. Kazemian, G. Varghese, and N. McKeown, "Header space analysis: Static checking for networks," in *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, 2012, pp. 113–126.

[21] A. Fogel et al., "A general approach to network configuration analysis," in *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, 2015, pp. 469–483.

[22] R. Beckett, A. Gupta, R. Mahajan, and D. Walker, "A general approach to network configuration verification," in *Proceedings of the 2017 Conference of the ACM Special Interest Group on Data Communication*, 2017, pp. 155–168.

[23] R. Birkner et al., "Config2spec: Mining network specifications from network configurations," in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, 2020.

[24] Forward Networks, *Forward Enterprise*, `https://www.forwardnetworks.com/`, 2024.

[25] SaltStack, *Salt: Configuration management and orchestration*, https://saltproject.io/, 2024.
[26] Cisco Systems, *Cisco Network Services Orchestrator (NSO)*, https://www.cisco.com/c/en/us/products/cloud-systems-management/network-services-orchestrator/index.html, 2023.
[27] I. Pepelnjak, *Netlab: Network configuration and testing lab*, https://netlab.tools/, 2024.

A Blueprint YAML Schema Reference

Listing 14. Complete blueprint schema (annotated).

```yaml
version: 1                      # integer, >= 1
imports: [<path>, ...]          # optional, relative file paths to merge

# -- Named groups --
groups:
  <name>: <cel_selector>        # shorthand: just a selector string
  # OR full form:
  <name>:
    selector: <cel_selector>
    kind: security_zone         # optional
    trust_level: <i32>          # required when kind is security_zone
    interface_selector: <cel_selector>  # optional, edge-level zone membership

# -- Named objects --
objects:
  addresses:
    <name>:
      prefixes: [<cidr>, ...]           # IPv4 CIDRs
      prefixes_v6: [<cidr>, ...]        # optional IPv6 CIDRs
  services:
    <name>:
      entries:
        - protocol: <string>            # e.g. "tcp", "udp"
          port: <u16>

# -- Named rules (CEL macros) --
rules:
  <name>: <cel_expression>      # referenced via use_rule check

layers:
  - name: <string>              # non-empty, layer identifier
    requires: [<string>, ...]   # optional, previously-declared layers
    primitives:

      # --- Topology tier ---

      # --- mesh_nodes ---
      - type: mesh_nodes
        selector: <cel_selector>
        mesh_type: full | hub_and_spoke | route_reflector
        hub_selector: <cel_selector>      # required for h&s
        spoke_selector: <cel_selector>    # required for h&s
        edge_properties: <json_object>    # deferred

      # --- build_protocol_layer ---
      - type: build_protocol_layer
        selector: <cel_selector>
        layer: <string>                   # target layer
        config: <json_object>             # deep-merged into clone
        clone_underlying: <bool>          # default: false
```

```yaml
# --- Allocation tier ---

# --- provision_ips ---
- type: provision_ips
  selector: <cel_selector>
  pool: <cidr_or_pool_name>
  subnet_size: <u8>                 # default: 30 (v4), 126 (v6)
  ipv6_pool: <cidr_or_pool_name>    # optional dual-stack
  ipv6_subnet_size: <u8>
  strategy: dense | sparse | hash   # default: dense
  pool_size: <u32>                  # for global pool carving
  vrf: <string>                     # default: "default"

# --- allocate_resources ---
- type: allocate_resources
  selector: <cel_selector>
  resource_type: <string>
  pool: <range_or_pool_name>
  strategy: dense                   # default: dense
  pool_size: <u32>

# --- global_pool ---
- type: global_pool
  name: <string>
  pool: <cidr>
  vrf: <string>

# --- global_resource_pool ---
- type: global_resource_pool
  name: <string>
  resource_type: <string>
  pool: <range>

# --- Policy tier ---

# --- build_routing_policy ---
- type: build_routing_policy
  selector: <cel_selector>
  policy_name: <string>
  statements:
    - name: <string>               # sequence number
      action: permit | deny
      match_condition: <string>    # optional
      set_action: <string>         # optional

# --- build_access_policy ---
- type: build_access_policy
  selector: <cel_selector>
  policy_name: <string>
  rules:
    - name: <string>
      action: permit | deny
      source_prefix: <string>      # optional
      destination_prefix: <string> # optional
      protocol: <string>           # optional
      source_port: <string>        # optional
      destination_port: <string>   # optional

# --- build_prefix_list ---
- type: build_prefix_list
  selector: <cel_selector>
  prefix_list_name: <string>
  entries:
```

```yaml
      - prefix: <cidr>
        action: permit | deny
        le: <integer>                # optional
        ge: <integer>                # optional

  # --- build_community_list ---
  - type: build_community_list
    selector: <cel_selector>
    community_list_name: <string>
    entries:
      - community: <string>          # e.g. "65001:1000"
        action: permit | deny

  # --- generate_safe_bgp_filters ---
  - type: generate_safe_bgp_filters
    selector: <cel_selector>
    prefix_list_name: <string>       # optional
    policy_name: <string>            # optional
    block_rfc1918: <bool>            # default: true
    permit_default: <bool>           # default: false

  # --- Overlay tier ---

  # --- build_vxlan ---
  - type: build_vxlan
    selector: <cel_selector>
    vni_base: <u32>
    mcast_group_base: <string>       # optional

  # --- build_evpn ---
  - type: build_evpn
    selector: <cel_selector>
    route_distinguisher_base: <string>
    route_target_base: <string>

  # --- Meta tier ---

  # --- tag_nodes ---
  - type: tag_nodes
    selector: <cel_selector>
    tags: {<key>: <json_value>}

  # --- map_hardware_inventory ---
  - type: map_hardware_inventory
    selector: <cel_selector>
    chassis_model: <string>
    slots: {<u32>: <linecard_model>}

  # --- assert_state (mid-pipeline checkpoint) ---
  - type: assert_state
    name: <string>
    severity: error | warning
    select: <cel_selector>
    check: <assertion_check>         # same types as top-level
    help: <string>                   # optional

  # --- conditional ---
  - type: conditional
    condition: <cel_expression>
    then_primitives: [<primitive>, ...]
    else_primitives: [<primitive>, ...]   # optional

  # --- custom_primitive ---
```

```yaml
      - type: custom_primitive
        name: <string>
        parameters: {<string>: <json_value>}
        primitives: [<primitive>, ...]

      # --- inject_secrets ---
      - type: inject_secrets
        selector: <cel_selector>
        secrets: {<metadata_key>: <env_var_name>}

      # --- wasm_plugin (requires --features wasm) ---
      - type: wasm_plugin
        selector: <cel_selector>
        plugin_path: <filesystem_path>    # path to .wasm binary

assertions:
  - name: <string>
    severity: error | warning
    select: <cel_selector>
    help: <string>                       # optional remediation hint
    check:
      type: field_exists
      field: <field_name>
      # OR
      type: min_count
      count: <integer>
      # OR
      type: max_count
      count: <integer>
      # OR
      type: min_edges
      count: <integer>
      # OR
      type: unique_per_group
      field: <field_name>
      group_by: <field_name>
      # OR
      type: field_in_cidr
      field: <field_name>
      cidr: <cidr>
      # OR
      type: custom_cel
      expression: <cel_expression>
      # OR
      type: use_rule
      name: <rule_name>                  # references rules: section
      # OR
      type: is_connected                 # no additional fields
      # OR
      type: is_acyclic                   # no additional fields
      # OR
      type: is_bipartite                 # no additional fields
      # OR
      type: reachability
      target_selector: <cel_selector>
      # OR
      type: match_schema
      schema: <json_schema_object>
      # OR
      type: zone_membership             # no additional fields
      # OR
      type: zone_policy_exists
      from_zone: <zone_name>
```

```
      to_zone: <zone_name>
      # OR
      type: no_shadowed_rules           # no additional fields
      # OR
      type: zone_pairs_covered          # no additional fields

# -- Blast radius policies --
blast_radius:
  - name: <string>
    select: <cel_selector>              # optional, defaults to all
    check:
      type: max_deletions
      count: <integer>
      # OR
      type: max_modifications
      count: <integer>
      # OR
      type: max_percentage_changed
      value: <float>                    # 0.0 to 100.0

# -- Target-specific transforms --
transforms:
  - name: <string>
    when: <cel_expression>              # optional device predicate
    rules:
      - match_expr: <cel_expression>    # selects stanzas
        apply:
          <field>: <cel_expression>     # computes new field value

# -- Hardware library --
hardware_library:
  chassis:
    - model: <string>
      slots: <integer>
      description: <string>
  linecards:
    - model: <string>
      ports: <integer>
      speed: <string>
```

B Selector DSL Grammar

Listing 15. Selector DSL grammar (informal EBNF).

```
selector     ::= "all()" | target "[" filter "]"
target       ::= "nodes" | "edges"
filter       ::= cel_expression

(* Surface syntax normalisations applied before CEL compilation *)
"="          -> "=="
"and"        -> "&&"
"or"         -> "||"
"not"        -> "!"
"&"          -> "&&"
"|"          -> "||"

(* CEL variables bound per entity *)
nodes:  id (int), type (string), layer (string), + all data_json fields
edges:  id (string), src (int), dst (int), layer (string),
        + all data_json fields

(* IP prefix length extraction *)
```

```
For any field with an IP/CIDR value "10.0.0.0/30":
  {field}_len = 30  (integer, bound automatically)

(* Undeclared references *)
Accessing a field that does not exist on a node evaluates to false
(not an error), enabling heterogeneous topologies.
```

C CLI Quick Reference

Table 34. CLI command summary.

Command	Synopsis
`generate`	Full pipeline: parse → transform → render → write
`validate`	Dry-run pipeline with optional template pre-flight
`plan`	Compute and display topology mutation plan (or config diff)
`inspect`	AST visualisation, Graphviz DOT export, or per-node trace
`ci-run`	Combined validate + plan + config diff for CI/CD
`import`	Import YAML topology to NTE archive
`lint`	Run design-rule assertions only
`fmt`	Standardise blueprint YAML formatting
`impact`	Calculate blast radius of a blueprint change
`report`	Generate Markdown architecture report
`export-clab`	Generate Containerlab simulation file
`sync`	Synchronise topology with NSoT (pull/push)